

INTRODUCCIÓN: ENTENDIENDO EL PROBLEMA

Los clientes o usuarios no pueden hacer el diseño por nosotros, pero sí pueden mostrarnos los problemas que tienen. No debemos construir lo primero que se nos ocurre, debemos entender bien el problema y poder entonces proponer una solución

Es importante:

- Conocer quién es el cliente
- Detectar los puntos de dolor del usuario
- Establecer objetivos
- Tratar de definir alguna métrica que indique que se resuelve dicho punto de dolor o cumple con el objetivo¹

Para convertir una visión en realidad, primero hay que averiguar si es un problema real para las personas y si la idea realmente resuelve ese problema. Para eso, es importante hablar y conocer a los usuarios y validar las suposiciones que nos hicimos.

Debemos dejar de trabajar sobre nuestros supuestos; dedicarle tiempo a validar si será realmente útil y valioso ya que es muy costoso construir algo para luego validarlo.

Build the thing right != Build the right thing

Product discovery

Entender el problema nos permite seleccionar qué es más importante o valioso para los usuarios. Sin embargo, siempre hay ciertos riesgos a tener en cuenta:

- Que lo que hagamos no sea valioso o no resuelve un problema relevante
- Que la gente no sepa utilizar el producto
- Que no contemos con el tiempo, habilidades o tecnología para hacerlo
- Que no sea rentable o sostenible

Design Thinking

Método para generar ideas innovadoras, que centra su eficacia en entender y dar solución a las necesidades reales de los usuarios. Hay varias fases:

- Empatizar: conocer el contexto en el que se va a trabajar y al cliente. Comenzamos con una profunda comprensión de las necesidades.
- Definir: detectamos oportunidades, y estudiamos los patrones de conducta, oportunidades de innovación, etc. Una vez está el problema definido, primero se piensan todas las soluciones (diverge) y luego se llega a una posible solución. Para que funcione bien, debe enmarcar un problema con un enfoque directo, ser inspiradora (la declaración) para trabajar en la solución, y describir qué piensa y hace el usuario
- Idear: el famoso Brain Storming. Proponemos soluciones según el contexto y las necesidades. La idea debe ser inspiradora y describir qué piensa y hace el usuario.
- Prototipar: seleccionamos la idea con la mejor propuesta de valor y la convertimos en un prototipo.
- Testear: el usuario forma parte de esta etapa, verificando que se cumple con lo solicitado. Cocreamos en base a la prueba y error

PENSAMIENTO DIVERGENTE: implica genera múltiples soluciones a un problema explorando diversas perspectivas y posibilidades. Fomenta la creatividad y la innovación al desafiar el pensamiento convencional y explorar soluciones no convencionales.

PENSAMIENTO CONVERGENTE: implica evaluar y seleccionar la mejor solución entre las opciones generadas a través del pensamiento divergente. Requiere análisis crítico y toma de decisiones para identificar la solución más efectiva al problema.

Todo esto tiene muchos beneficios:

- Reduce el riesgo.
- Mejora la calidad.
- Aborda las necesidades reales.
- Fomenta la innovación al explorar múltiples vías para el mismo problema.
- Convierte los problemas en oportunidades.
- Fomenta el trabajo en equipo (multidisciplinario)

- Mejora la empatía en la cultura empresarial.
- Se obtienen resultados tangibles y consistentes

Una buena idea es pensarlo así:

Usuario

Necesita

Problema

Porque Insight

El Brain Storming hay cuatro reglas fundamentales: toda crítica está prohibida; toda idea es bienvenida; tantas ideas como sea posible; el desarrollo y asociación de las ideas es deseable.

OUTCOMES VS OUTPUTS

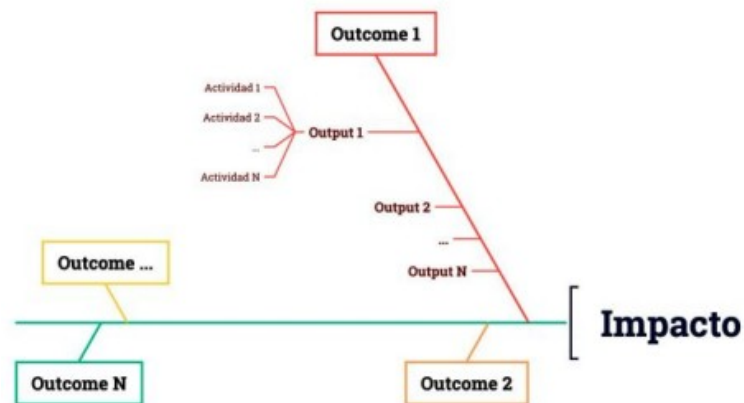
Muchas veces medimos el éxito por lo que entregamos (outputs), en lugar del efecto real (outcome). Lo importante no es lo que entregamos, sino cómo eso cambia la manera en que las personas actúan y generan valor.

OUTPUT: es un entregable o actividad concreta, algo que construimos o hacemos. El problema de enfocarnos en los outputs es que podemos llegar a crear funcionalidades que nadie usan hacemos reportes que nadie lee, definimos procesos que no generan un cambio real, entre otros.

OUTCOME: cambio de comportamiento que impulsa resultados de negocio. Por ejemplo, no es hacer una presentación de ventas, sino que los clientes avancen rápido con el proceso de compra.

Modelado del problema.

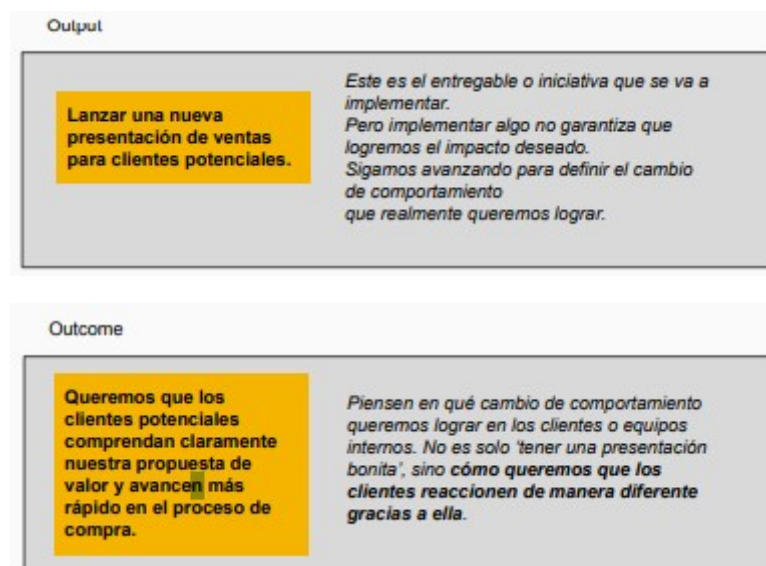
- RECURSOS: planificar los recursos
- ACTIVIDADES: llevar a cabo un conjunto de actividades.
- OUTPUTS: si todo sale según lo planeado, se crea el output.
- OUTCOMES: si todo funciona según lo planeado, logramos el outcome.
- IMPACTO: esto, a su vez, contribuye directamente al impacto que buscamos.



Si gestionamos basados en **outputs**, estos no pueden generar valor ya que estamos basando la gestión en lo que hacemos y no en los beneficios que buscamos. Si gestionamos por **impacto**, no somos lo suficientemente específicos ya que estos pueden tener múltiples raíces. Por lo tanto, gestionando **outcomes** buscamos comportamientos específicos del cliente que aportan a los resultados de negocio (que son los impactos). Por lo tanto, definir outcomes en términos de comportamientos humanos crea una forma de trabajar totalmente centrada en el cliente y el usuario.

Entonces, ¿cómo podemos estar seguros de que lo que estamos haciendo realmente entregará valor? No podemos. Por eso, es necesario plantear hipótesis, la cual tiene dos partes: lo que creemos (la idea) y la evidencia que nos permitirá validarlo.

Ejemplos:



Métrica

Tiempo promedio desde el primer contacto hasta la toma de decisión del cliente

Si logramos este cambio de comportamiento, ¿cómo lo mediríamos? ¿Qué número o indicador debería mejorar?

Hipótesis

Hipótesis 1: Si estructuramos la presentación enfocándonos en los problemas del cliente en lugar de solo hablar de nuestros servicios, aumentará el interés y la conversión.

Hipótesis 2: Si incluimos casos de éxito reales y métricas claras, los clientes potenciales tendrán más confianza y estarán más dispuestos a avanzar.

No basta con hacer la presentación. Necesitamos probar hipótesis sobre qué acciones harán que realmente genere impacto. Escriban al menos 1 y máximo 3 hipótesis sobre cómo podríamos lograr el outcome que definieron.

Hipótesis 3: Si enviamos la presentación de forma personalizada con un seguimiento estratégico, aumentará la cantidad de clientes que piden una reunión.

DISEÑO DE CÓDIGO



Objetivos a la horade escribir código: mantenibilidad, simplicidad, claridad, flexibilidad y legibilidad

- Es preferible nombres claros o comentarios. También usar nombres que revelen su intención, incluyendo variables, constantes y funciones.
- No dejar valores fijos, usar constantes con nombres claros.
- Para los métodos, usar siempre la misma palabra (get, set, fetch, receive, etc.).
- Tener métodos pequeños que hagan una sola cosa. Además, que tengan un solo nivel de abstracción.
- Evitar los switch y que se reciban argumentos.
- Es preferible usar polimorfismo, o crear nuevas funciones.
- The Boy Scout Rule: dejar el código mejor que lo que lo encontraste.
- Es bueno usar comentarios, y los hay de varios tipos: legales, informativos (por ejemplo, qué hace una función), explicación de una intención, clarificación (por ejemplo, en los tests), advertencias de consecuencias, etc.
- Malos comentarios: redundancia, ruido, marcadores de posición, al cerrar una llave, código comentado, etc.
- Excepciones: usar excepciones en vez de código de error; tratar de no retornar Null
- Tests unitarios: F.I.R.S.T. (Fast, Independent, Repeatable, Self-Validating, Timely). Siempre escribir casos de prueba aislados (probar solo una cosa en un solo caso de prueba); utilizar solo un

assert por caso de prueba; utilizar una convención de nombres para los casos de prueba

CRITERIOS DE BUEN DISEÑO

El análisis es acerca del qué, el diseño es acerca del cómo.

Criterios de mal diseño:

- **RIGIDEZ:** es difícil de cambiar. Un cambio sencillo requiere muchas modificaciones.
- **FRAGILIDAD:** un sistema frágil es aquel donde cambiar una cosa rompe otras inesperadas.
- **INMOVILIDAD:** no se puede reutilizar código aunque lo necesites, no porque sea imposible, sino porque está tan entrelazado con otras partes que extraerlo es demasiado costoso o arriesgado.
- **VISCOSIDAD:** ocurre cuando es más fácil hacer las cosas mal que hacerlas bien.

Los diseños se vuelven rígidos, frágiles e inmóviles por la incorrecta dependencia entre módulos.

Criterios de buen código:

- **COHESIÓN:** mide qué tan relacionadas y enfocadas están las responsabilidades dentro de un módulo, clase o función. Alta cohesión significa que cada módulo hace una sola cosa bien, y una clase con alta cohesión es fácil de entender, mantener y probar.
- **BAJO ACOPLAMIENTO:** el acoplamiento mide qué tanto depende un módulo de detalles internos de otros. Bajo acoplamiento significa que los módulos están conectados de manera débil y controlada, causando que cambiando uno no impacte tanto sobre los demás. Se logra usando interfaces, inyección de dependencias y respetando el principio de inversión de dependencias (D de SOLID)

PRINCIPIOS DE DISEÑO

SINGLE RESPONSIBILITY PRINCIPLE: principio de responsabilidad única. Cada clase/módulo debe encargarse de una sola cosa

```
// ✖ Mal diseño: hace demasiadas cosas
class UsuarioService {
    void guardarUsuario(Usuario u) { ... } // lógica de negocio
    void enviarEmail(Usuario u) { ... }    // notificaciones
    void generarReporte(Usuario u) { ... } // reportes
}

// ✔ Buen diseño: una responsabilidad por clase
class UsuarioService { void guardarUsuario(Usuario u) { ... } }
class EmailService    { void enviarEmail(Usuario u) { ... } }
class ReporteService  { void generarReporte(Usuario u) { ... } }
```

OPEN-CLOSED PRINCIPLE: el código debe estar abierto a la extensión pero cerrado a la modificación. Se debe poder extender el comportamiento sin modificar el código existente, y se logra con herencia, interfaces o composición.

```
// ✔ Abierto a extensión: se pueden crear nuevas figuras sin modificar el código existente
interface Figura {
    double area();
}

class Circulo implements Figura {
    double radio;
    Circulo(double r) { this.radio = r; }
    public double area() { return Math.PI * radio * radio; }
}

class Rectangulo implements Figura {
    double ancho, alto;
    Rectangulo(double a, double b) { this.ancho = a; this.alto = b; }
    public double area() { return ancho * alto; }
}

class CalculadoraArea {
    double calcularTotal(List<Figura> figuras) {
        return figuras.stream().mapToDouble(Figura::area).sum();
    }
}
```

LISKOV SUBSTITUTION PRINCIPLE: los objetos de una clase derivada deben poder sustituir a los de su clase base sin alterar el comportamiento esperado, es decir, una subclase no debe romper las reglas de su superclase.

✖ Ejemplo clásico (mal diseño):

```
java

class Ave { void volar() { ... } }

class Pinguino extends Ave {
    @Override void volar() { throw new UnsupportedOperationException(); }
}
```

✖ Un `Pinguino` es un `Ave`, pero no puede volar → viola LSP.

✓ Solución:

```
java

interface Ave { }
interface AveVoladora extends Ave { void volar(); }

class Gorrión implements AveVoladora { public void volar() { ... } }
class Pinguino implements Ave { }
```

INTERFACE SEGREGATION PRINCIPLE: es mejor tener interfaces pequeñas y específicas que interfaces grandes y generales. Evitar obligar a una clase a implementar métodos que no necesita.

```
// ✖ Mal diseño
interface Trabajador {
    void programar();
    void testear();
    void diseñar();
}

class Tester implements Trabajador {
    public void programar() { /* no aplica */ }
    public void testear() { ... }
    public void diseñar() { /* no aplica */ }
}

// ✓ Buen diseño
interface Programador { void programar(); }
interface QA { void testear(); }
interface Diseñador { void diseñar(); }

class Tester implements QA {
    public void testear() { ... }
}
```

DEPENDENCY INVERSION PRINCIPLE: los módulos de alto nivel no deben depender de módulos de bajo nivel, ambos deben depender de abstracciones. Se logra usando interfaces e inyección de dependencias.

```
// ✖ Mal diseño: UsuarioService depende directamente de EmailNotificador
class EmailNotificador {
    void enviar(String msg) { ... }
}
class UsuarioService {
    private EmailNotificador notificador = new EmailNotificador();
    void registrar(String nombre) {
        notificador.enviar("Bienvenido " + nombre);
    }
}

// ✔ Buen diseño: depende de una abstracción
interface Notificador { void enviar(String msg); }

class EmailNotificador implements Notificador {
    public void enviar(String msg) { ... }
}
class SmsNotificador implements Notificador {
    public void enviar(String msg) { ... }
}

class UsuarioService {
    private Notificador notificador;
    UsuarioService(Notificador n) { this.notificador = n; }
    void registrar(String nombre) {
        notificador.enviar("Bienvenido " + nombre);
    }
}
```

AGILIDAD, SCRUM, XP Y KANBAN

MARCO CYNEFIN

COMPLEJO Es lo desconocido. Son las prácticas emergentes (experimentar/detectar/responder). Es volátil, complejo, incierto y ambiguo	COMPLICADO Puede ser conocido y no es familiar. Son las buenas prácticas (detectar/analizar/responder)
CAÓTICO No puede ser conocido. Son las prácticas novedosas (actuar/detectar/responder)	CLARO Es lo conocido y familiar. Son las mejores prácticas (detectar/clasificar/responder)

AGILIDAD

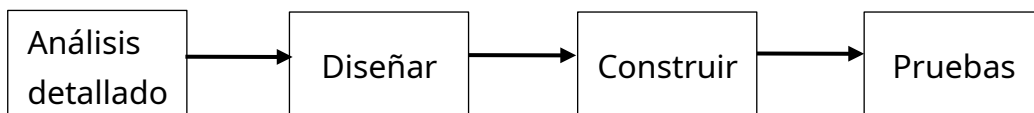
- NO ES
 - Una **METODOLOGÍA** porque no es algo que pretenda indicarnos qué hacer paso a paso.
 - Ir más **RÁPIDO**, aunque entregar valor temprano sí es una de las promesas ágiles.

- **MULTITASKING**, aunque implique gestionar un contexto complejo, incierto y cambiante.
- **SÍ ES**
 - Una **FORMA DE HACER** que nos invita a experimentar y aprender en pasos pequeños.
 - Una **FORMA DE SER** en la que nos ayudamos, nos cuidamos y mejoramos en nuestro entorno.
 - Un **CAMINO** al cual una vez entramos, muy posiblemente sigamos recorriendo.
 - Una **MENTALIDAD** que tienen quienes se enfocan en dar valor, trabajando colaborativamente.

SER	HACER
Mindset, valores y principios	Frameworks y metodologías; técnicas y herramientas

TRADICIONAL VS ÁGIL

Tradicional:

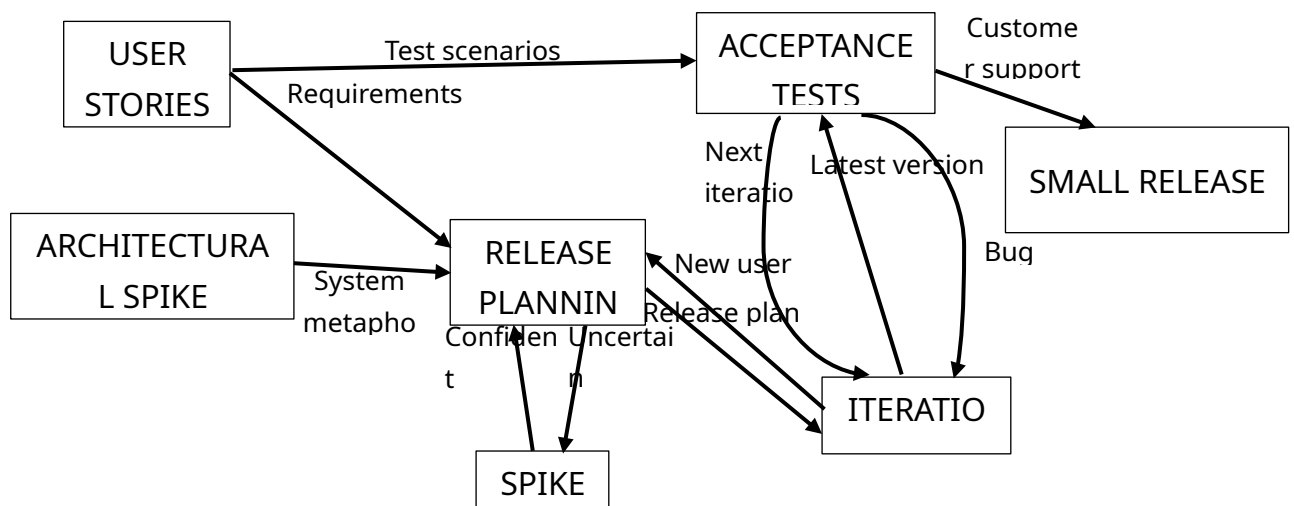
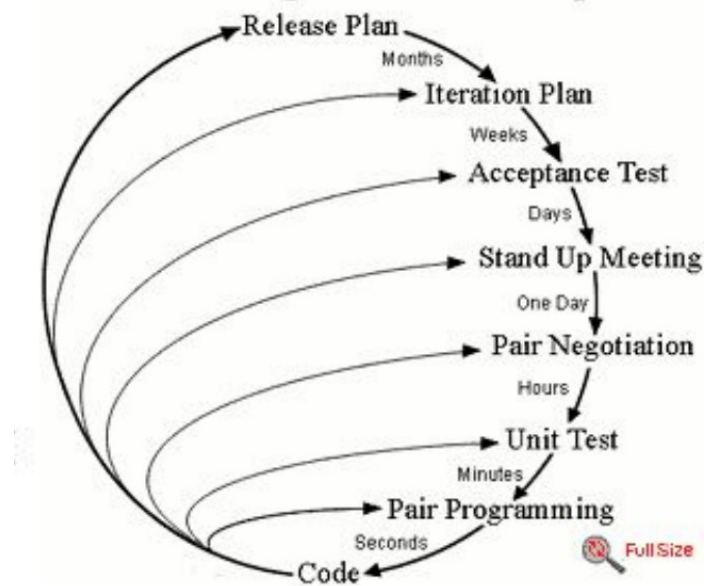


Ágil: es igual al tradicional, solo que se van a priorizar ciertas tareas para tratarlas primero. Se da por medio de iteraciones, en donde en cada iteración se realizan los mismos pasos que en la metodología tradicional. Igualmente, antes de la primera iteración sucede una reunión en donde sucede la priorización de tareas.

¿Por qué son ciclos? Porque permite que el malentendido se detecte temprano. Además, hay ciclos de mejora en dos casos: el producto (resuelve el problema, es lo que se pidió, etc.), y la forma de trabajo (recibe feedback, cambios, etc.).

XP (EXTREME PROGRAMMING)

Planning/Feedback Loops

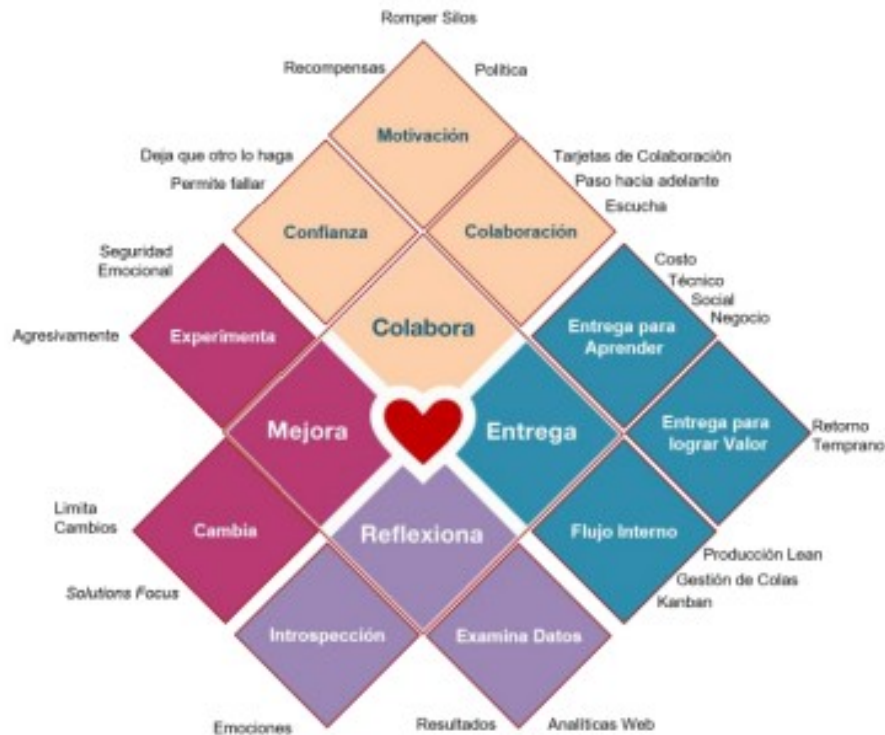


CORAZÓN DE LA AGILIDAD

La agilidad puede ser expresada en cuatro palabras:

- **COLABORA:** queremos mejorar la confianza y la motivación.
- **ENTREGA:** tiene aspectos internos y externos. En los internos encontramos el desarrollo incremental, los cuellos de botella, los límites de trabajo en progreso (WIP), Kanban y otros procesos. En la parte externa encontramos los temas de entregar-para-obtener-aprendizaje versus entregar-para-obtener-valor
- **REFLEXIONA Y MEJORA:** suelen estar entrelazados ya que la reflexión rara vez se hace bien. La reflexión se divide en dos partes: recopilación de información subjetiva, emocional, generalmente

sobre el equipo y el proceso, e información objetiva desde el análisis de datos sobre el producto y su recepción por usuarios y compradores.



PATRONES DE DISEÑO

Son soluciones comunes a problemas que se ven con frecuencia. Facilita muchísimo la comunicación. Hay varios tipos de patrones: los creacionales (tienen que ver en el cómo se van a crear los objetos), los estructurales (cómo podemos ensamblar los diferentes objetos y clases para poder crear una estructura que permita que las cosas sigan con una dinámica fluida, eficiente, etc.) y los de comportamiento (cómo se comportan los objetos).

STRATEGY

La idea es poder identificar las diferentes formas que tenemos de hacer algo, encapsularlas, y poder hacer que el resto del programa sea independiente de cada una de estas estrategias, y que sean intercambiables en distintos puntos del programa.

Este patrón mejora **Single Responsibility Principle** en el aspecto de que cada “estrategia” se encarga de realizar solo lo suyo. También mejora

Open/Closed Principle ya que para implementar una nueva “estrategia” solo basta con crear una nueva clase.

STATE

Las acciones que se pueden realizar o los errores que puedan surgir dependen del estado actual del sistema. Los pasos a seguir son: primero se crea la interfaz, y luego se crea una clase por cada estado.

Este patrón mejora el **Single Responsibility Principle** porque cada estado hace una cosa, y hay un estado para cada situación. También mejora **Open/Closed Principle** porque si agrego un nuevo estado, solo tengo que implementarlo desde la interfaz.

STATE VS STRATEGY.

STRATEGY: se enfoca en intercambiar diferentes algoritmos a métodos que logran el mismo objetivo.

STATE: gestiona diferentes comportamientos según el estado actual del objeto.

En **State**, los estados casi siempre conocen a los otros estados y pueden iniciar transiciones. En cambio, en **Strategy**, una “estrategia” casi nunca conocerá a las demás.

SCRUM

Es un marco de trabajo ligero que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.

- Es un **framework**, no una metodología, ya que proporciona herramientas y prácticas flexibles.
- Es **versátil**, ya que tiene la estructura justa para ser incorporada por equipos, y prácticas para ser adecuada a sus necesidades.
- **Pilares del Scrum**: transparencia (siempre se puede ver el proceso), inspección (todos pueden inspeccionar el trabajo para obtener información y poder tomar decisiones), y adaptación (adaptarse al ambiente de trabajo)

ARTEFACTOS

Representan trabajo o valor. Están diseñados para maximizar la transparencia de la información clave. Por lo tanto, todos los que inspeccionan tienen la misma base para la adaptación.

- **PRODUCT BACKLOG**: pila donde se guarda todo lo que queremos implementar o charlar. Lo más prioritario tiene que estar arriba.
- **SPRINT BACKLOG**: luego de una “ceremonia” (sprint planning), lo obtenemos. Estas ideas ya están planificadas y entendidas. Probablemente queden tareas sin resolver.
- **INCREMENTO**: cualquier cosa concreta que aporte valor. Es importante que se establezca la definición de “hecho”.

ROLES

Son las capacidades y responsabilidad de las personas que ocupan cada espacio. Dentro de un equipo de Scrum no hay sub-equipos ni jerarquías, sino que es una unidad cohesionada de profesionales enfocada en un objetivo a la vez, el objetivo del Producto.

- **PRODUCT OWNER (PO)**: define qué se debe construir y prioriza el trabajo. Es la voz del cliente, y se encarga de maximizar el valor del producto. Tiene una visión dual: Stakeholders y Development Team.
 - **RESPONSABILIDADES**: conocer mercado y comportamiento de clientes/usuario; es el representante de los Stakeholders para

conseguir una buena visión de los objetivos; priorización de los objetivos.

- o CARACTERÍSTICAS: conocedor del negocio; habilidades de comunicación; capacidad de negociación; decisivo; accesible; orientado al logro
- **SCRUM MASTER (SM):** facilita el proceso y ayuda al equipo a mejorar continuamente. Se asegura de promover Scrum tal como se define. No es un Project Manager ya que no se encarga de gestionar al equipo, sino que es un líder servicial.
 - o RESPONSABILIDADES: explicar el marco de trabajo a los nuevos miembros; guiar al equipo en la adopción de buenas prácticas ágiles; garantizar que se respeten las ceremonias de Scrum; eliminar obstáculos al progreso del equipo; facilita la resolución de conflictos dentro del grupo; ayuda a que el equipo tome decisiones sin depender de un jefe.
 - o CARACTERÍSTICAS: mentalidad de mejora continua; liderazgo servicial.
- **EQUIPO DE DESARROLLO:** construye el producto y entrega valor en cada iteración. Produce el incremento de producto que se obtiene al final del Sprint. Autoorganizado y multidisciplinario. No hay un líder que asigne tareas.
 - o RESPONSABILIDADES: comprende claramente los requerimientos; producir entregables de alta calidad; identificar oportunidades de mejora.
 - o CARACTERÍSTICAS: autoorganizados; independientes; colaboradores; expertos técnicos; orientados a los objetivos.

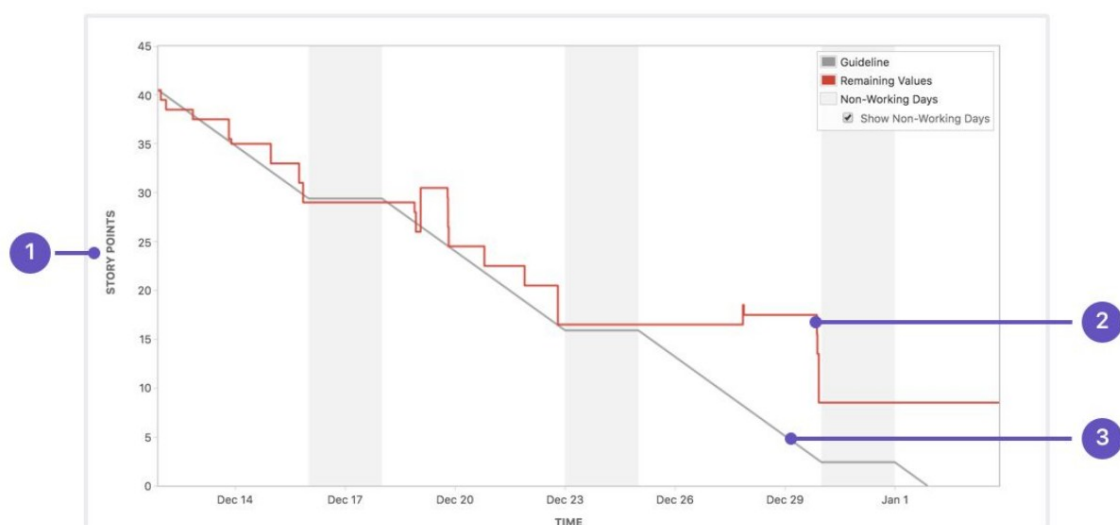
	P O	S M	DEVELOPERS	STAKEHOLDER S
Definir la meta del sprint	X		X	
Eliminar impedimentos que afectan al trabajo del equipo		X		
Armar las historias de usuario	X		X	
Alinear el producto con la estrategia de la organización	X			X
Facilitar las reuniones		X		
Comunicar progreso a la organización	X			
Apurar al sector IT para que le		X		

den al equipo acceso a una base datos				
Probar el producto			X	X

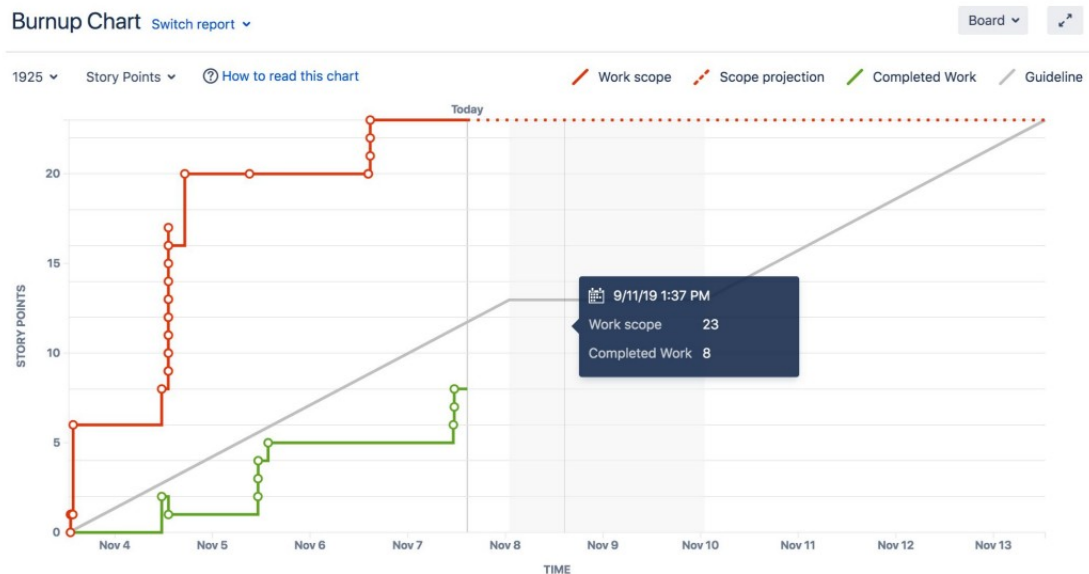
	PO	S M	DEVELOPERS	STAKEHOLDER S
Negociación	X			
Capacidad técnica avanzada			X	
Visión estratégica del producto	X			X
Mentalidad de mejora continua		X		
Empatía con usuarios	X		X	
Comunicación clara y constante	X	X	X	
Capacidad de adaptación al cambio	X	X	X	
Disposición para dar y recibir feedback	X	X	X	

MÉTRICAS

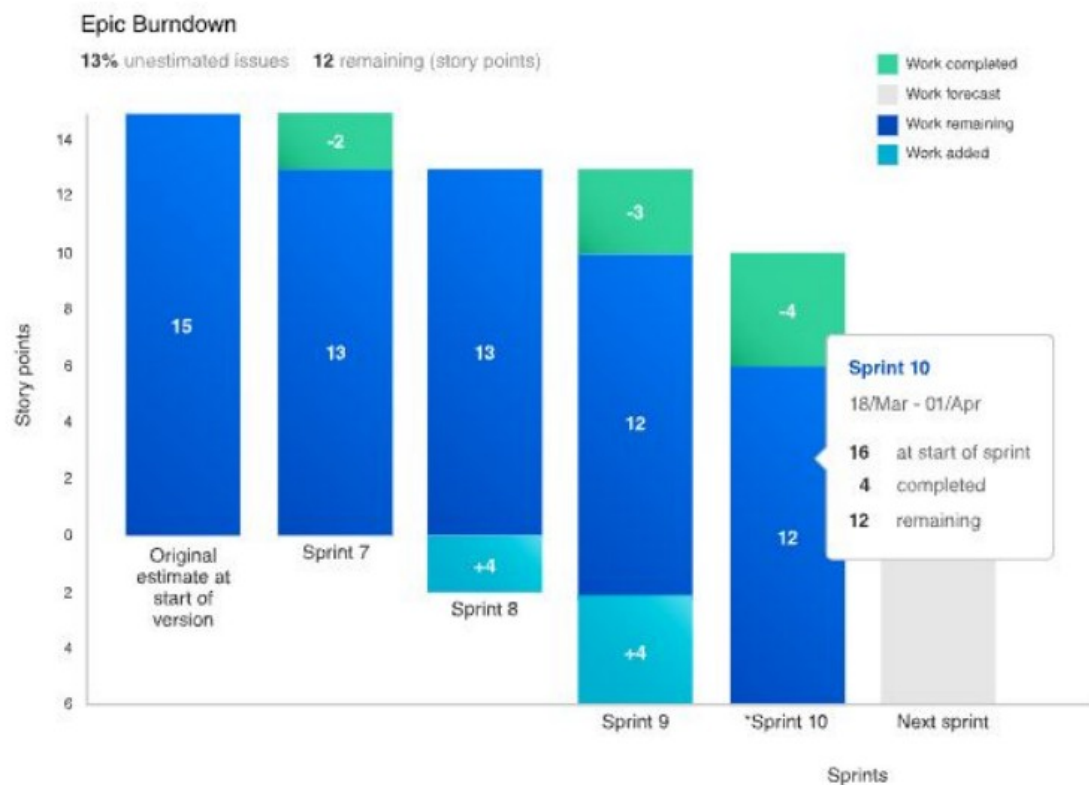
BURNDOWN CHART: representa el trabajo pendiente que tenemos en el sprint contra el tiempo restante. La línea gris es lo ideal. El eje Y está medido en story points (forma de estimar cuánto trabajo vamos a darle a una tarea). Cuando la línea naranja sube es porque se dividieron tareas o se hicieron mal. Lo que no se realiza pasa al siguiente sprint.



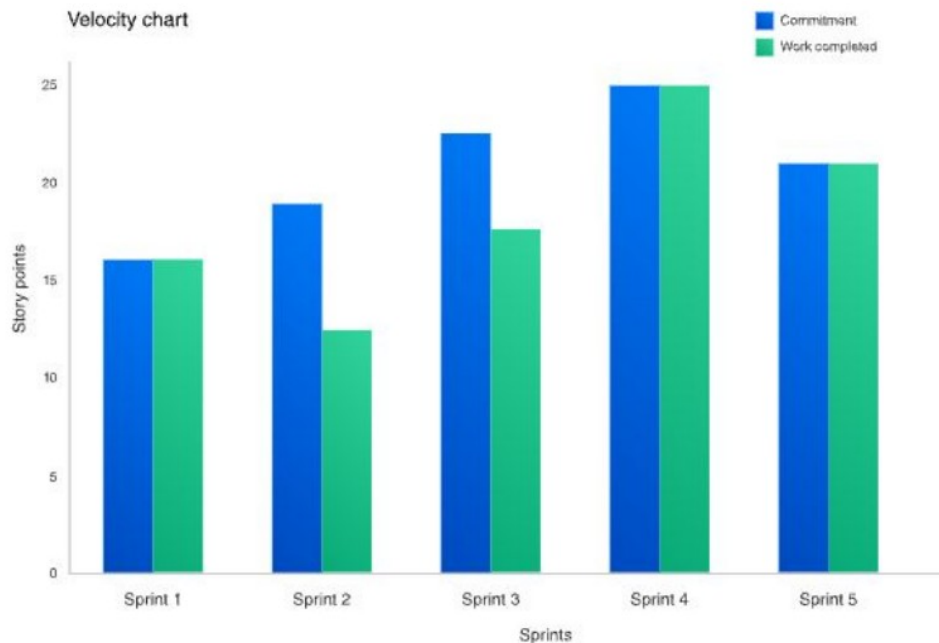
BURNUP CHART: muestra cómo va creciendo el trabajo que vamos completando (línea verde) contra el producto terminado (línea roja). Se usa más para analizar el proyecto, no tanto para el sprint.



EPIC BURNDOWN CHART: una épica es una forma de agrupar tareas. Este gráfico no se mide tanto por sprint sino por épicas en concreto (se van midiendo esas épicas por sprint, es decir, cómo va avanzando en cada sprint).



VELOCITY CHART: mide los story points que puede completar el equipo en determinado sprint, y también marca los que se comprometieron a hacer. Se mide en sprints (mide cuántos story points se pueden meter por sprint). No sirve para estimar cuánto le queda al proyecto, sino que sirve para predecir.



CEREMONIAS

Son reuniones que le dan ritmo al trabajo a lo largo del sprint.

- **SPRINT PLANNING:** indica cómo y qué vamos a hacer en el sprint. Participan el PO, el SM y los Devs. La idea es agarrar los stories del producto backlog que se harán en el sprint. Dura aproximadamente 4 horas (sprint de 2 semanas). Resulta en el sprint goal y en el sprint backlog (estimado y asignado). Hay que hacerle preguntas al PO para luego escribir criterios de aceptación a esas preguntas. Si hay ambigüedades, el equipo debe preguntar y validar, pero cómo última instancia puede definir suposiciones. Se puede subdividir en tareas y estimar cada una. Si no hay suficiente claridad, no se debe estimar.
- **DAILY SCRUM O STANDUP MEETING:** organiza el trabajo de las siguientes 24 horas. Participan los Devs y el SM, duran aproximadamente 15 minutos, y su resultado es el que todos conozcan la situación del sprint y qué es lo que tienen que hacer en

el día. Algunas preguntas que se pueden hacer son: ¿qué hice ayer?, ¿qué voy a hacer hoy?, ¿tengo algún bloqueante o problema?

- **SPRINT REVIEW O DEMO:** se hace al final del sprint. Se valida el trabajo realizado y se ajusta el trabajo futuro. Participan el PO, el SM, los Devs y los Stakeholders. Dura aproximadamente 1 hora, y resulta en el feedback de los Stakeholders y en modificaciones en el backlog.
- **SPRINT RETROSPECTIVE O RETRO:** se busca mejorar la forma de trabajo. Participan los Devs y el SM. Duran aproximadamente 1 hora, y resulta en action ítems para el siguiente sprint.

Ceremonia	Propósito	Asistentes	Duración
Sprint Planning	- Qué hacer - Cómo hacerlo	Devs PO SM	4 hs
Daily	Organizar el trabajo del día	Devs SM	15 min
Sprint Review	- Validar trabajo realizado - Ajustar Backlog	Devs PO SM Stakeholders	1h
Sprint Retrospective	Mejorar la forma de trabajo	Devs SM	1h

PRODUCT DISCOVERY

Muchos clientes piden un MVP (Minimun Viable Product) casi completo para mantener la imagen, otros piden que simplemente funcione (para uso temprano y sacar más ganancias). Este MVP se divide en 3 etapas: el EST (Earliest Tetable Product), el EUP (Earliest Usable Product) y el ELP (Earliest Lovable Product). Por eso, para conseguir el MVP hay que ir a dea poco.

PRODUCUT DISCOVERY

Se basa en comprobar hipótesis, y generalmente sucede durante la etapa de desarrollo. Es muy probable que a medida que se va desarrollando, la funcionalidad del proyecto decaiga ya que se encuentran nuevas

funcionalidades, complicaciones, mejoras a aplicar, etc., que hace que se tenga que retroceder el desarrollo para centrarse en estas cosas.

Un producto nunca está “listo”, sino que está en constante evolución, siempre apuntando a entregar más valor a los clientes.

A medida que se desarrolla el producto, uno se va dando cuenta qué es lo que sirve y qué es lo que no, es decir, hay un Dual Track entre Discovery y Development. Es importante el Discovery inicial, aunque no alcanza con solo ese, sino que hay que hacerlos constantemente.

Mentalidad de Product Discovery

- COLABORACIÓN Y APRENDIZAJE EN EL EQUIPO DE PRODUCTO: todo el equipo de producto conoce y aprende del PD, participando en las decisiones sobre la funcionalidad a crear, evitando conflictos de opiniones, desarrollando y manteniendo un entendimiento compartido con artefactos de visualización)
- USER CENTERED DESIGN: tener comunicación directa con los usuarios y priorizar las necesidades de los mismos durante todas las fases del desarrollo, desarrollando experiencias
- HUMILDAD INTELECTUAL: tolerancia a la incertidumbre (no hay certezas sino hipótesis a validar), experimentos (tolerancia al error y a cambiar de parecer), construir un ambiente seguro e inclusivo de respeto a las opiniones de todo el equipo.

PROBLEMA VS SOLUCION: no es tan lineal, sino que puede ocurrir que cuando estemos en la solución surjan nuevos problemas que tendremos que resolver.

ENTREGA: temprana y frecuente; darle valor; priorización; generar ingresos; obtener retroalimentación.

HISTORIAS DE USUARIO

USER PERSONA

Representación ficticia pero basada en datos reales de un tipo de usuario. Sirve para: guiar decisiones de diseño y desarrollo (orientar el diseño para que se adopte mejor a las decisiones del usuario), alinear al equipo en torno a quién es el usuario final, y hacer foco en segmentos específicos (no

tratar de abarcar a todos los usuarios, sino a los más representativos). Se usa para conocer al cliente y quiénes serán nuestros usuarios finales.



MAPA DE EMPATÍA

Herramienta visual que ayuda a profundizar en el entendimiento del usuario, respondiendo a la pregunta: “Cómo piensa, siente y experimenta esta persona su problema?”. Tiene más detalles de la información que User Persona y nos ayuda a ponernos en el lugar del usuario. Sus beneficios son: generar empatía real con el usuario, identificar motivaciones y frenos para usar el producto, y es un puente entre la investigación y definición de requerimientos.



HISTORIAS DE USUARIO

Los valores de las metodologías ágiles son:

1. Individuos e interacciones sobre procesos y herramientas.

2. Software funcionando sobre documentación exhaustiva.
3. Colaboración con el cliente sobre negociación contractual.
4. Respuesta ante el cambio sobre seguir un plan

Son una descripción de la funcionalidad del sistema, desde el punto de vista de un usuario.

Como <persona>
quiero <función>
para <objetivo>

- Se enfocan en el valor provisto, dando pie a discutir el requisito: no son el requisito en sí.
- Se centra en el punto de vista de un rol que usará o se verá afectado por la solución.
- Habla del requisito en un lenguaje que tenga significado para ese rol.
- Ayuda a aclarar la verdadera razón del requisito.
- Ayuda a definir requisitos de alto nivel sin necesidad de entrar en detalles de bajo nivel demasiado pronto.

¿CÓMO ESCRIBIR HISTORIAS DE USUARIO DE CALIDAD?

I	ndependent	Separable de otras tareas
N	egotiable	No es innecesariamente rígido
V	aluable	Aporta valor al cliente/usuario
E	stimable	Posible aproximar su costo
S	mall	Desarrollable en una iteración
T	estable	Verificable (al menos en principio)

Calidad Sintáctica	Bien formadas	Atómicas
	Mínimas	
Calidad Semántica	Conceptualmente acertadas	Orientadas al problema
	Sin ambigüedades	Sin conflictos
Calidad Pragmática	Usar oraciones completas	Estimables
	Únicas	Uniformes
	Independientes	Completas

ÉPICAS: historias de usuario que se pueden descomponer en más historias de usuario. Deseamos subdividirlas hasta llegar a tareas de tamaño razonables.

3 CS: CARD (describe la intención del usuario), CONVERSATION (los interesados se comunican para refinar las historias, descubriendo y documentando requisitos) y CONFIRMATION (criterios de aceptación).

CRITERIOS DE ACEPTACIÓN: condiciones específicas que deben cumplirse para aceptar trabajo realizado. Se da por tarea, es decir, cada tarea tiene sus propios criterios de aceptación. Deben ser claros, concisos, verificables, independientes y orientados al problema.

Dado que <contexto>
cuando suceda <evento>
entonces <consecuencia>

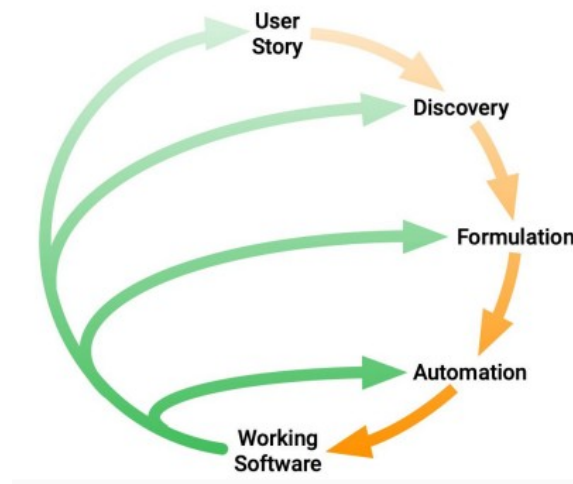
Given <contexto>
When <evento>
Then <consecuencia>

BEHAVIOUR DRIVEN DEVELOPMENT (BDD)

Es una forma de trabajo para equipos de software que cierra la brecha entre las personas de negocio y las personas técnicas. Fomenta la colaboración entre roles, se trabaja iteraciones rápidas y pequeñas para aumentar la retroalimentación y el flujo de valor, y se produce documentación del sistema que se verifica automáticamente contra el comportamiento del sistema.

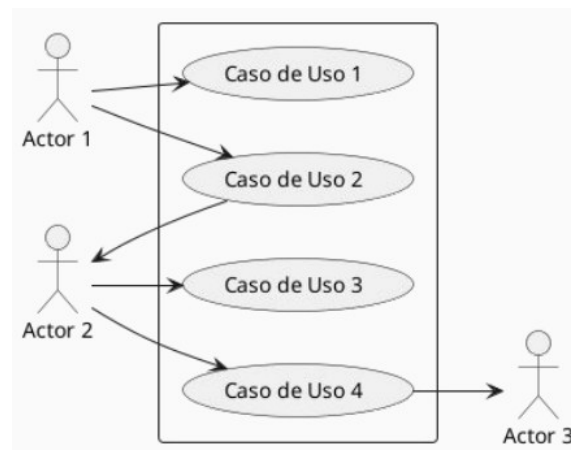
En el día a día hay tres etapas:

1. **DISCOVERY:** tomamos un pequeño cambio próximo en el sistema y revisamos ejemplos concretos de la nueva funcionalidad.
2. **FORMULATION:** documentamos esos ejemplos de una manera que pueda ser automatizada y verificamos el acuerdo.
3. **AUTOMATION:** finalmente, implementamos el comportamiento descrito por cada ejemplo, comenzando con una prueba automatizada que guíe el desarrollo.



CASOS DE USO

Descripción de una serie de acciones realizadas por un sistema, que generan un resultado observable de valor para un actor en particular. Están compuestos por un escenario principal y un conjunto de escenarios alternativos.



EJEMPLO:

Consultar Precio de un Producto	
El cliente obtiene información del precio de un producto	
Precondiciones	

Cliente	Sistema
1. Ingresa el código de producto	
	2. Informa el precio del producto
Postcondiciones	

Escenarios alternativos	
2.1: Si el código de producto no corresponde a un producto registrado, el sistema informa que desconoce el producto	

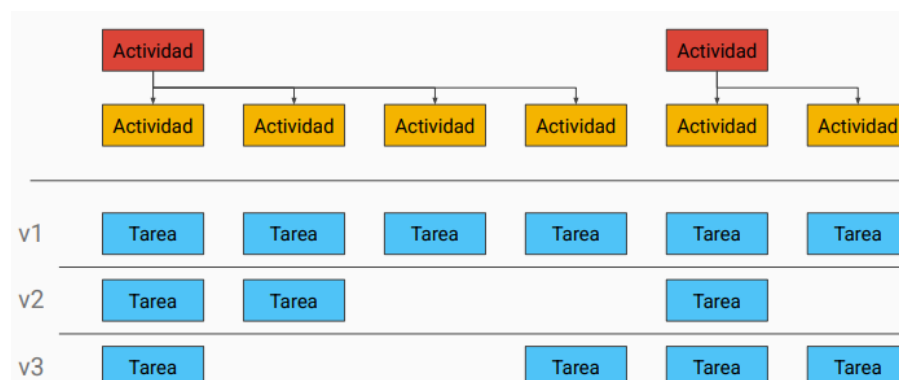
También existen versiones más compactas:

Consultar Precio de un Producto
1. El cliente ingresa el código de producto 2. El sistema informa el precio del producto 2.1. Si el código de producto no corresponde a un producto registrado, el sistema informa que desconoce el producto
Consultar Precio de un Producto
El cliente ingresa el código de producto. El sistema informa el precio del producto o que desconoce el producto

¿CÓMO ENCONTRAR HISTORIAS?

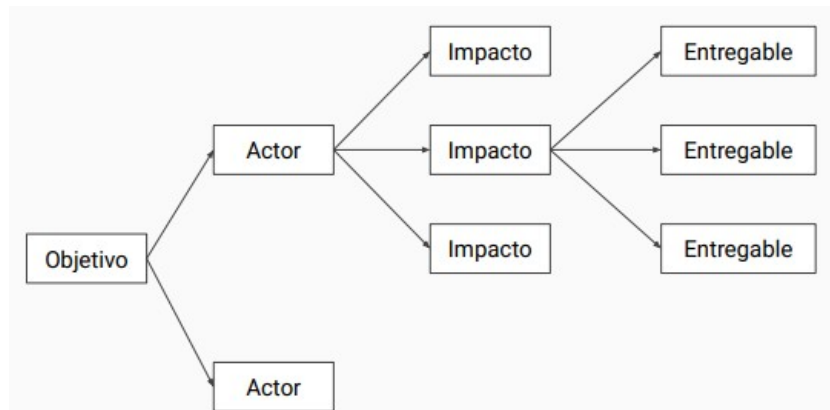
Hay que establecer los límites del sistema, identificar a los actores involucrados en el sistema, y buscar objetivos y actividades por medio de un User Story Mapping o Impact Mapping.

USER STORY MAPPING:



Hay una tarea con un altísimo nivel de actividad, la cual tiene más actividades que la componen. A su vez, cada actividad tiene sus propias tareas, que se van a ir realizando por fases o versiones.

IMPACT MAPPING



Está orientado a los objetivos de negocio y al impacto que se busca generar. Los entregables son las historias de usuario.

EJEMPLO:

Goal	Actor	Impact	Deliverable
Increase revenue by 15% in the next year	Ann	Spend more time on the platform	Propose fast shipping Trendy looks section Personalized content
	Dave	Increase average order value	Mails for latest sales Discount for second product Membership deals

MODELOS DE DOMINIO

Un modelo es una representación simplificada de la realidad que permiten visualizar, entender y especificar. Estos sirven como guía y de documentación.

COMPLEJIDAD DEL DESARROLLO DE SOFTWARE

Complejidad = Complejidad Esencial (relativo a la solución del problema) + Complejidad Accidental (cosas que hicimos que impactaron en el tiempo de la solución pero que no hacía falta que estuvieran ahí)

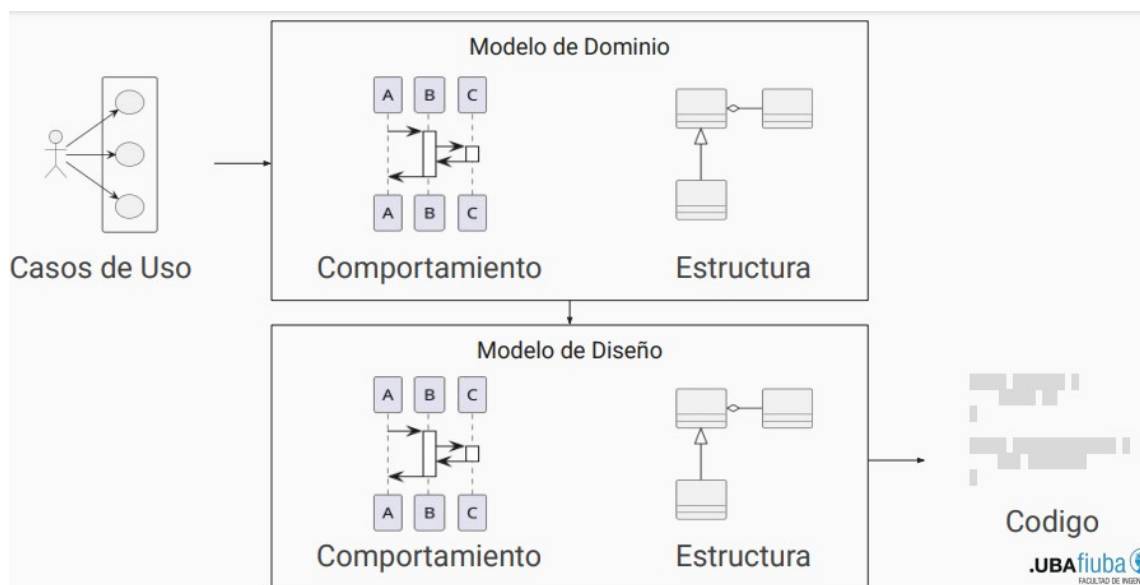
Complejidad Esencial = Complejidad Problema + Complejidad Solución.

Para resolver el problema es necesario entenderlo antes de empezar a pensar una solución.

MECANISMOS: descomposición (descubrir), abstracción (pensar) y establecer jerarquías (inventar).

MODELO DE DOMINIO

Tiene como objetivo entender en detalle el negocio y sus reglas. Utiliza patrones de análisis. En cambio, el modelo de diseño busca implementar una solución al modelo planteado en el análisis teniendo en cuenta las restricciones impuestas por los requerimientos no funcionales, y utiliza patrones de diseño.



TÉCNICAS:

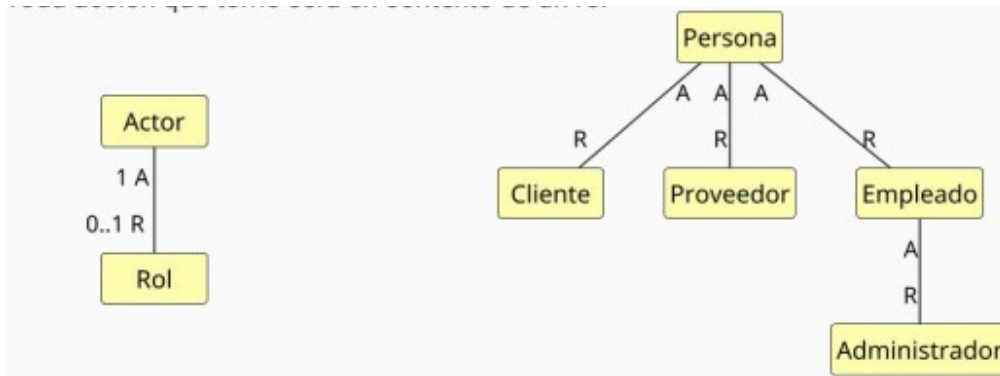
- HACER DIRECTAMENTE EL MÓDELO DE DISEÑO Y/O CÓDIGO: sacrifica la posibilidad de validar el modelo con un experto del dominio. Es posible si los desarrolladores son expertos del dominio. Tiene un uso más común.

- **ANÁLISIS SINTÁCTICO:** buscar sustantivos y verbos significativos en casos de uso, minutas, etc. Algunas categorías útiles son: actores (humanos y no), objetos físicos, lugares, eventos, procesos, etc.
- **CRUD:** Create, Read, Update, Delete. Toda entidad debe ser creada y leída en alguna funcionalidad. Usualmente, también pueden modificarse o eliminarse. Si falta una operación, entonces falta una funcionalidad.
- **PATRONES DE ANÁLISIS:** reutilizar las soluciones que alguien más ya pensó. Son soluciones comunes a problemas comunes, utilizan vocabulario común, ya fueron probados, y permiten dedicar esfuerzo a otros propósitos.
- **STREAMLINED OBJECT MODELING:** libro de Jill Nicola, habla de patrones de colaboración (forma de relacionar las entidades), reglas de negocio (formas de restringir cómo se pueden relacionar las entidades) y recetas para traducir patrones y reglas a código. Está especializado para dominios típicos de sistemas Enterprise.

Patrón Fundamental	Patrones
Genérico - Específico	• Actor - Rol • Item - Item Específico • Transacción Compuesta - Line Item
Entero - Parte	• Gran Lugar - Lugar • Ensamble - Parte • Contenedor - Contenido • Grupo - Miembro
Específico - Transacción	• Rol - Transacción • Lugar - Transacción • Item Especifico - Transacción • Item Especifico - Line Item • Transaccion - Transacción Cronologica

.UB.

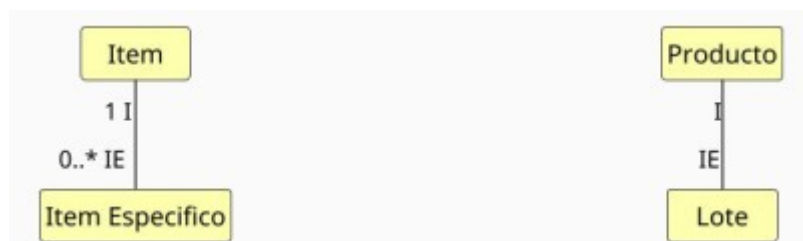
ACTOR - ROL: el actor puede o no tener un rol, pero este siempre corresponde a un actor.



PATRONES DE TRANSACCIÓN SIMPLE: una transacción conoce quién la realiza, dónde y sobre qué aplica.



ITEM - ITEM ESPECÍFICO: el ítem describe la información que es común en todas las instancias concretas.



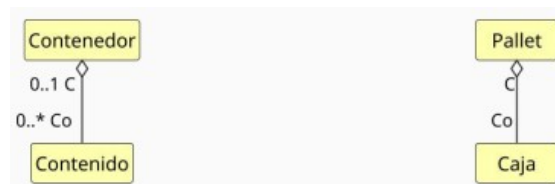
ENSAMBLE - PARTE: un ensamble se compone necesariamente de una o más partes. No pueden existir por separado.



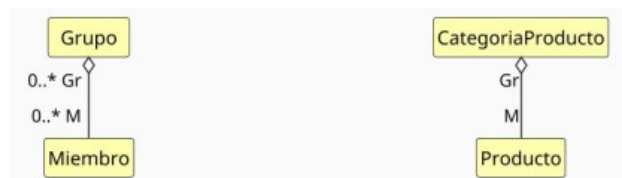
GRAN LUGAR – LUGAR: un gran lugar sirve como contenedor de sus lugares.



CONTENEDOR – CONTENIDO: un contenedor es un recipiente para otras cosas. Pueden existir por separado.

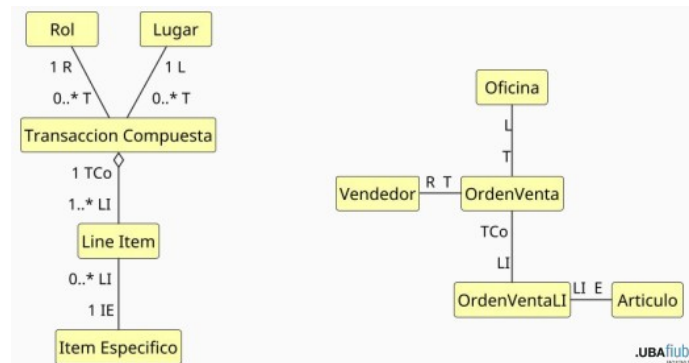


GRUPO – MIEMBRO: los grupos modelan colecciones y clasificaciones de cosas, personas y lugares. Cada miembro puede estar dentro de múltiples grupos, y pueden existir por separado.

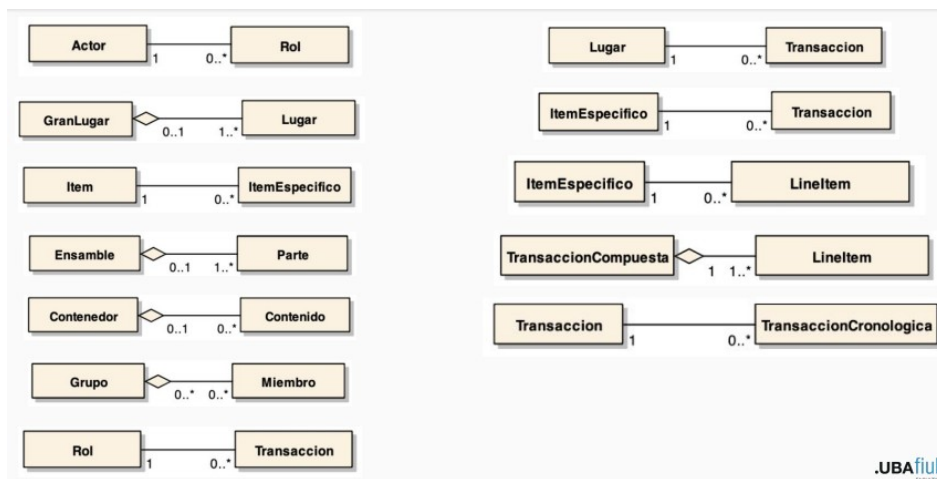
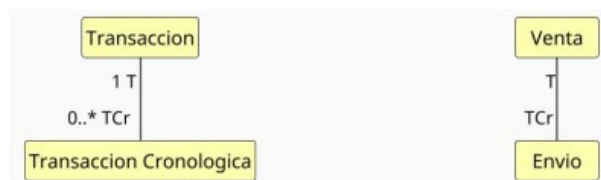


.PATRONES DE TRANSACCIÓN COMPLETA: involucra múltiples ítems. Para cada uno de los ítems específicos que se van a relacionar vamos a crear una entidad intermedia (Line Item) y va a relacionar dos patrones (Transformación Completa con Line Item y Line Item con Ítem Específico).

La idea del Line Item es que puede verificar las reglas de negocio (verifica las restricciones).



TRANSACCIÓN - TRANSACCIÓN CRONOLÓGICA: sirve para modelar interacción que siguen a interacciones anteriores (en el ejemplo, no puede existir un envío si antes no ocurrió una venta)



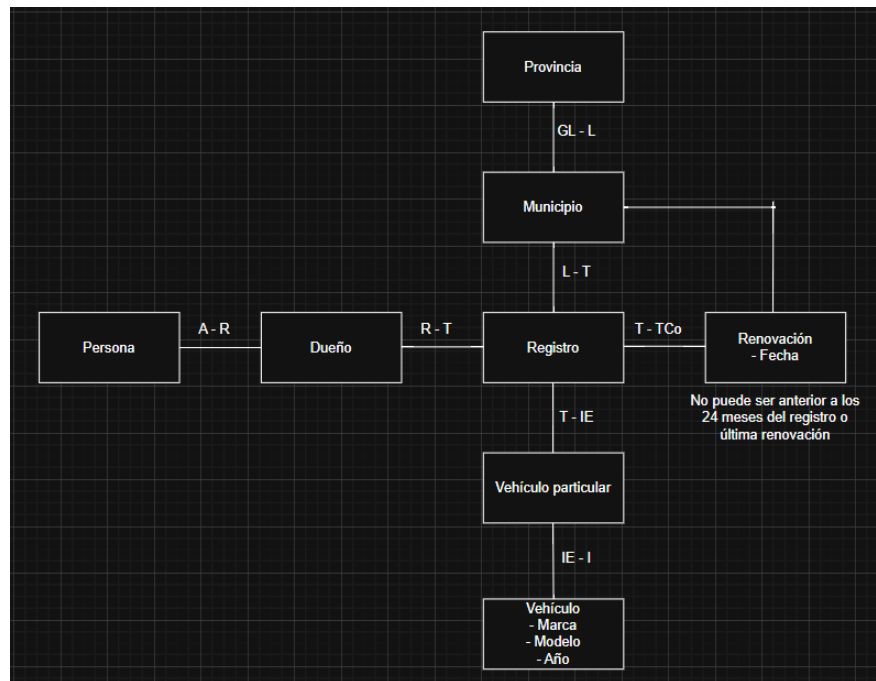
REGLAS DE NEGOCIO

Son restricciones que gobiernan las acciones dentro de un dominio (antes de que una entidad empiece o deje de colaborar con otra, hay que verificar ciertas cosas que dependen del dominio). Se traducen como verificaciones a realizar antes de modificar las colaboraciones entre los distintos objetos del modelo. Se las ubica en el modelo para evitar que la dinámica de los

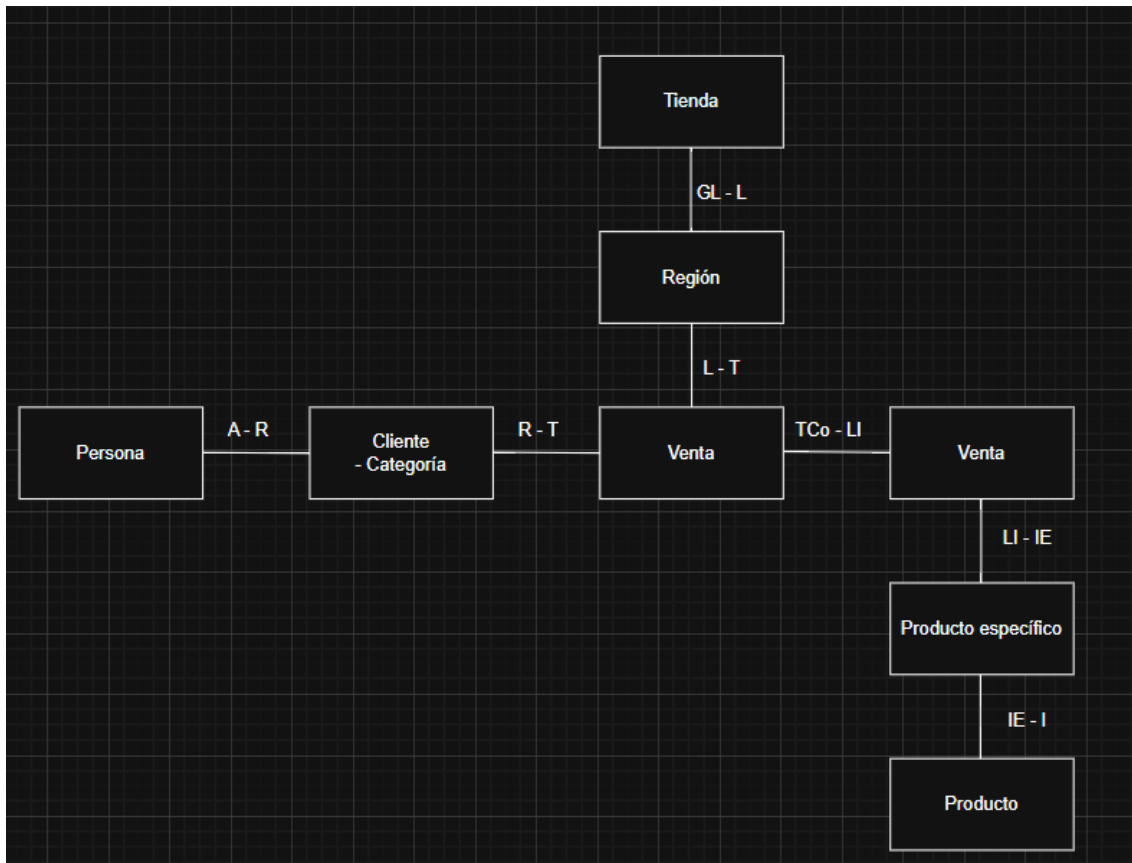
objetos sea externa. Está asociada al objeto que dispone de más información relevante.

EJERCICIOS

El registro de un vehículo se realiza en un municipio de una provincia. Involucra a un vehículo en particular y a su dueño. Cada vehículo tiene una descripción que incluye su marca, modelo y año. Cada registro puede renovarse múltiples veces, en el mismo o diferente municipio, pero no pueden repetirse antes de los 24 meses.



Una tienda vende productos a clientes de diferentes categorías, manteniendo estadísticas de venta por región, producto y categoría.

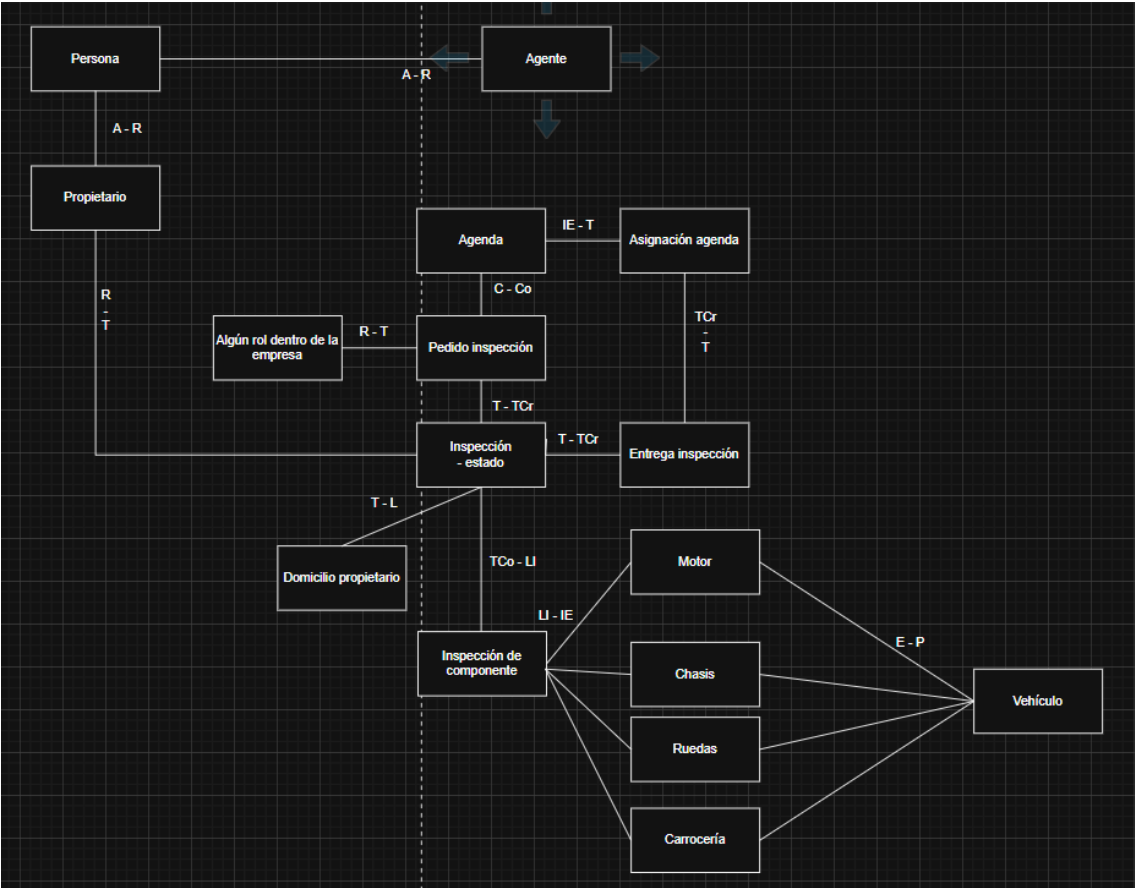


Se desea desarrollar una aplicación para la carga de datos de inspecciones de automotores para una empresa de seguros.

La empresa posee una dotación de 100 agentes inspectores. Cada agente de seguros, portador de un teléfono o notebook según se defina y que la empresa le entregará recibirá a través de la aplicación la agenda del día con los datos de los automotores a inspeccionar. Cada agenda está identificada con el número de empleado del agente el cual consta de veintiun caracteres alfanuméricos, este será verificado al momento de ser recibida. La agenda posee los datos de cada automotor a inspeccionar y los de su propietario. Las agendas son diarias y cada una no podrá incluir más de 6 inspecciones, además la aplicación no podrá recibir una agenda si previamente no envió a la empresa todas las inspecciones de la agenda del día anterior.

El agente se traslada al domicilio del propietario y realiza la inspección del automotor agendado para ser asegurado. Al realizar cada inspección el agente deberá registrar la marca, el modelo y los números de serie de cada componente (chasis, ruedas, motor, carrocería) y verificar y registrar su estado de conservación. Cada componente podrá estar en buen, regular o mal estado. En el caso de no poder ser identificado deberá marcarse como no legible.

Las inspecciones podrán realizarse de una vez o podrán empezarse y continuarse para ser terminadas en otro momento. Las inspecciones podrán ser canceladas y también marcadas como de imposible realización registrando la razón. Una vez completada una inspección, ésta podrá ser enviada a la empresa o almacenada para ser enviada a posteriori. Las inspecciones no realizadas o canceladas no podrán ser enviadas a la empresa.



DIAGRAMAS UML

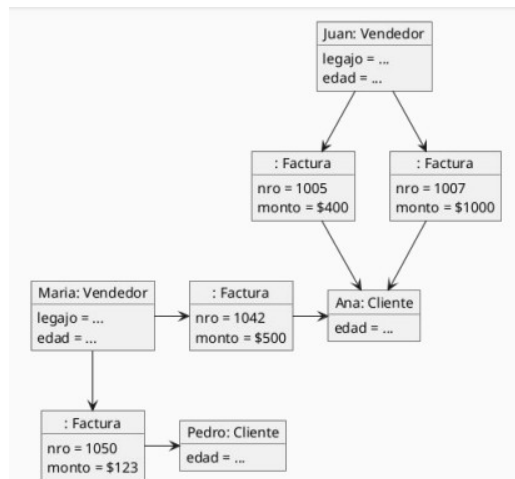
UML, lenguaje unificado de modelado, es una forma de representar visualmente sistemas de software complejos. Facilita la comprensión de ideas y sistemas complejos mediante la visualización. Se utiliza principalmente en la etapa de diseño.

Formaliza cómo representar ciertos aspectos del software en diagramas.

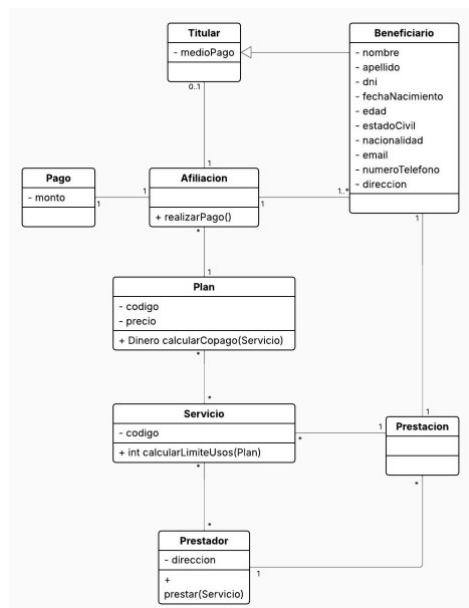
TIPOS DE DIAGRAMAS

ESTRUCTURA:

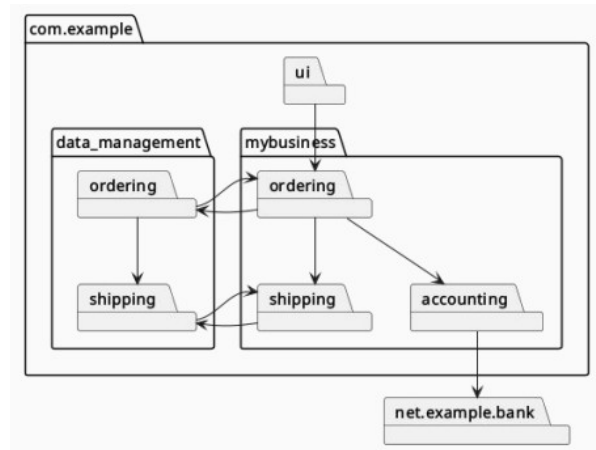
- **OBJETOS:** muestra las instancias particulares de las clases, es decir, los objetos, y las relaciones entre ellos.



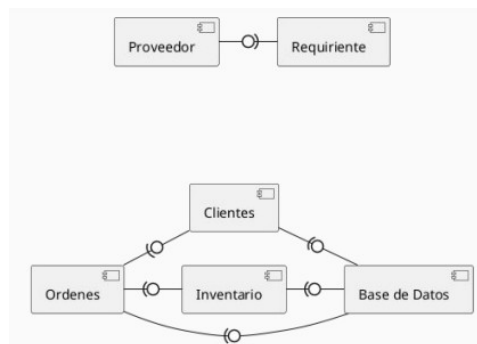
- **CLASES:**



- **PAQUETES:** da la estructura de más alto nivel de cómo interactúan las partes del sistema. Relación entre los paquetes. Cada paquete representa una parte del diseño. Lo mejor es agrupar por concepto de dominio, no por tipo de clase.

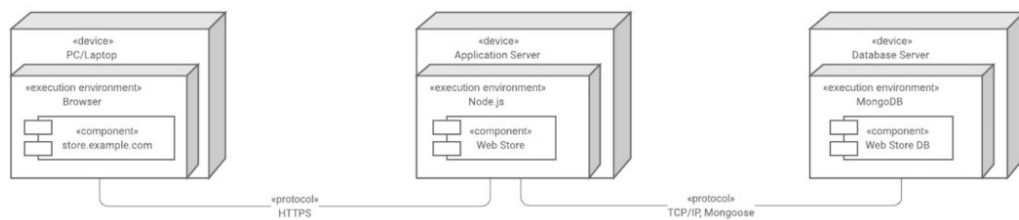


- **COMPONENTES:** hay subsistemas que proveen cosas, y otros que necesitan esas cosas. Apunta más al comportamiento en tiempo de ejecución.



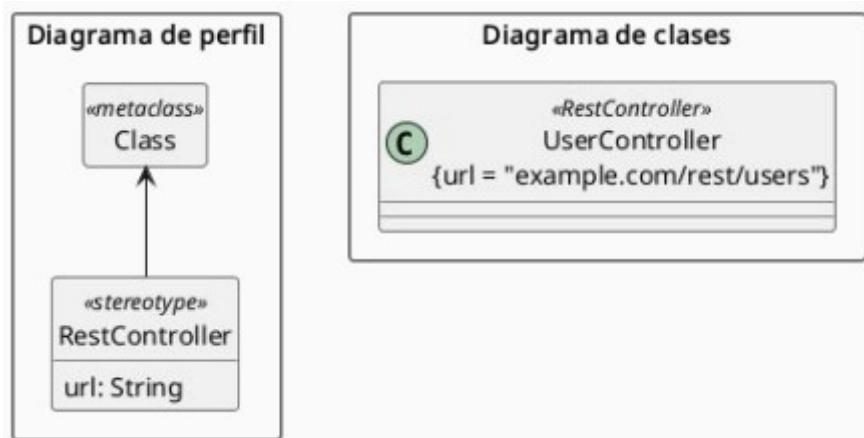
- **DESPLIEGUE:** se muestran dispositivos con entornos de ejecución y componentes (código ejecutable). Esto permite ver dónde está corriendo nuestro código y cómo se comunican los distintos dispositivos. Si los dispositivos no comparten nada, entonces son dos diagramas diferentes.

Si tengo varias ejecuciones en simultáneo en el mismo dispositivo, lo que se hace es que ese dispositivo tendrá varias “cajitas” con los entornos.



- **PERFIL:** una metaclass es un mecanismo de extensión. La metaclass decide la implementación (por ejemplo, con los atributos, métodos o una clase padre). Esto no tuvo éxito, así que surgió el concepto de estereotipo, que también es un mecanismo de extensión, con la diferencia que se aplica a un elemento de cualquier otro diagrama y modifica cómo se interpreta (ejemplo, @RestController hará algo adicional en runtime).

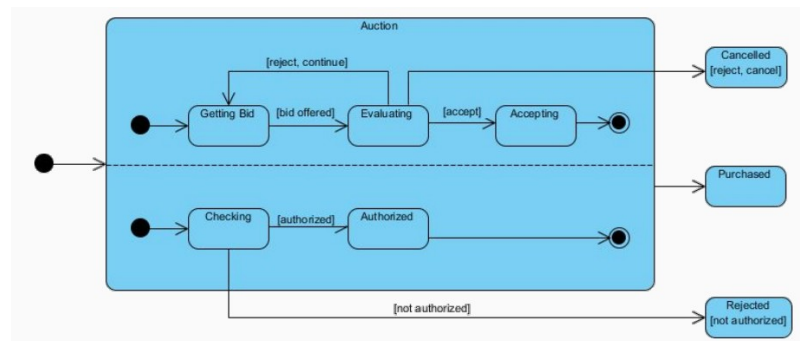
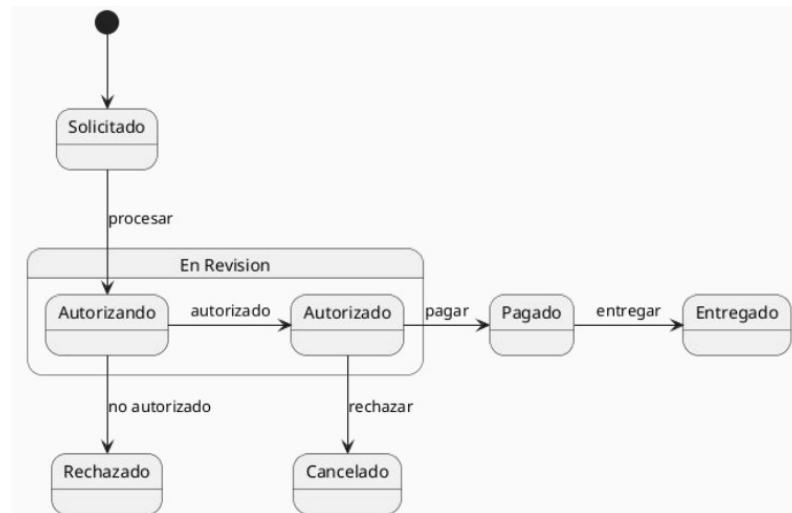
Un perfil UML formaliza un conjunto de estereotipos, tags y restricciones que se pueden aplicar en otros diagramas. Usamos el estereotipo metaclass para los elementos donde aplica: Class, Association, Attribute, Operation, Component, etc. En Java vas a usar una metaclass, en UML un estereotipo.



- **ESTRUCTURA COMPUESTA**

COMPORTAMIENTO:

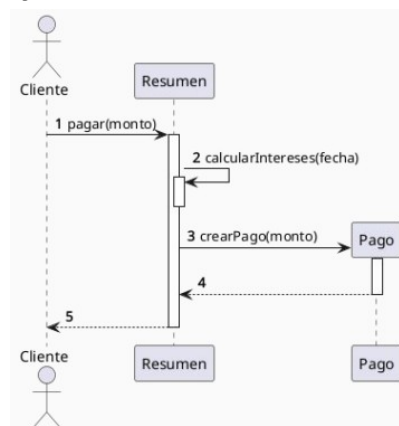
- **ACTIVIDAD:** muestra como es el flujo del programa. Hay distintos actores que participan que tienen sus propias acciones. Las líneas negras significan que las acciones se van a realizar en paralelo.



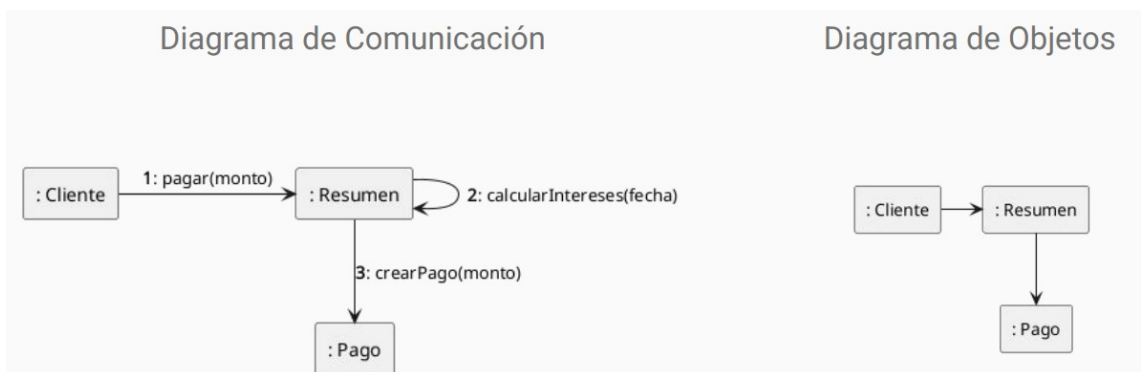
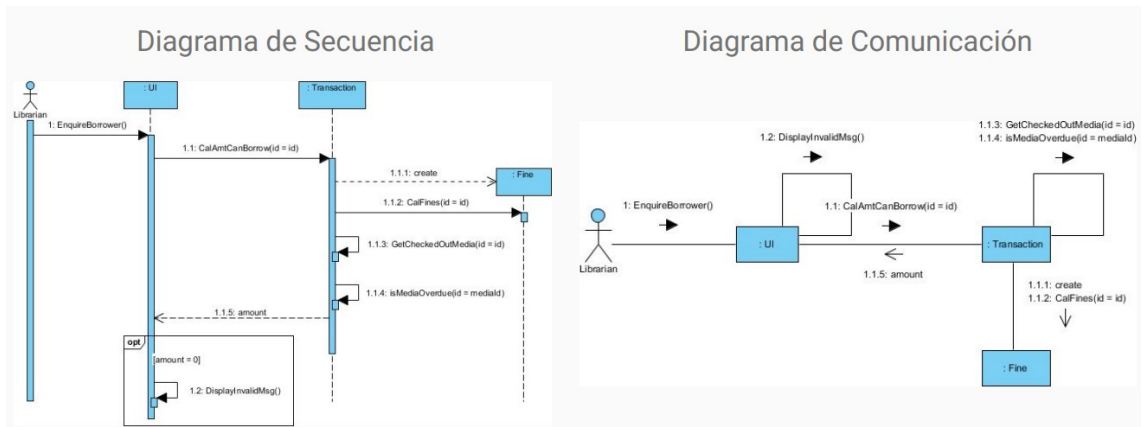
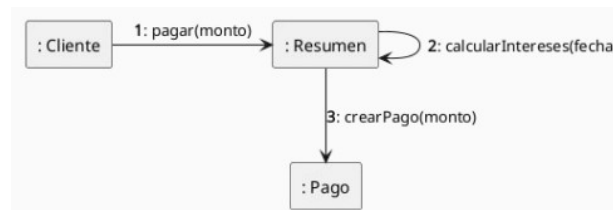
- INTERACCIÓN

OTROS:

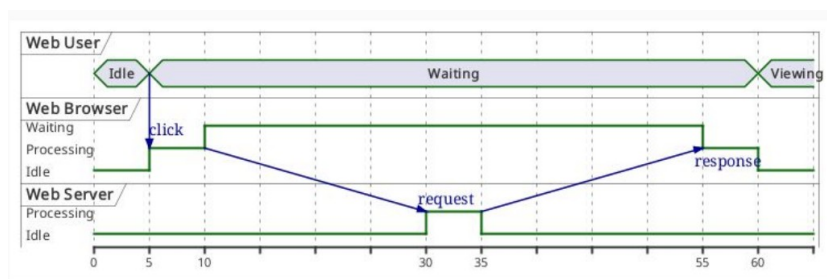
- SECUENCIA: hay líneas de vida de distintas entidades, las cuales muestran en que momentos existen ciertos actores en el flujo del programa. Este es más específico que el diagrama de actividad, ya que se usan métodos y nombres concretos.



- COMUNICACIÓN: parecido al diagrama de secuencia, con la diferencia que muestra, en las acciones, el número de orden.



- TIEMPOS: muestra cuánto tiempo se está en cada estado. Las flechas azules muestran las interacciones.

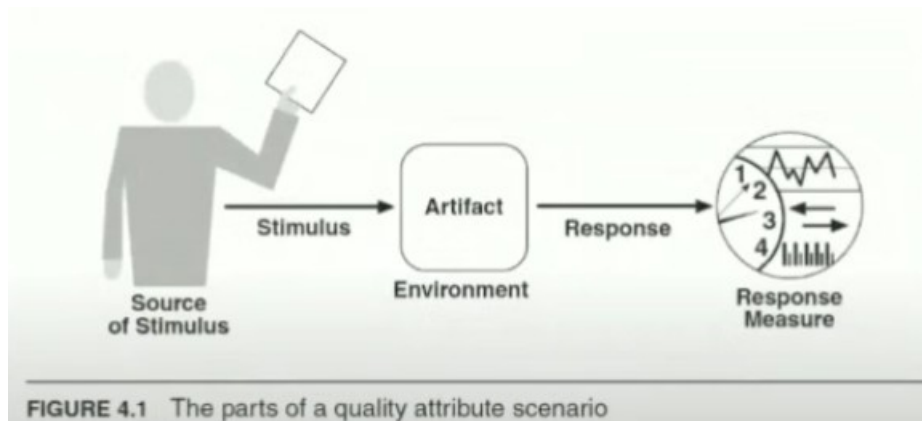


- INTERACCIÓN GLOBAL

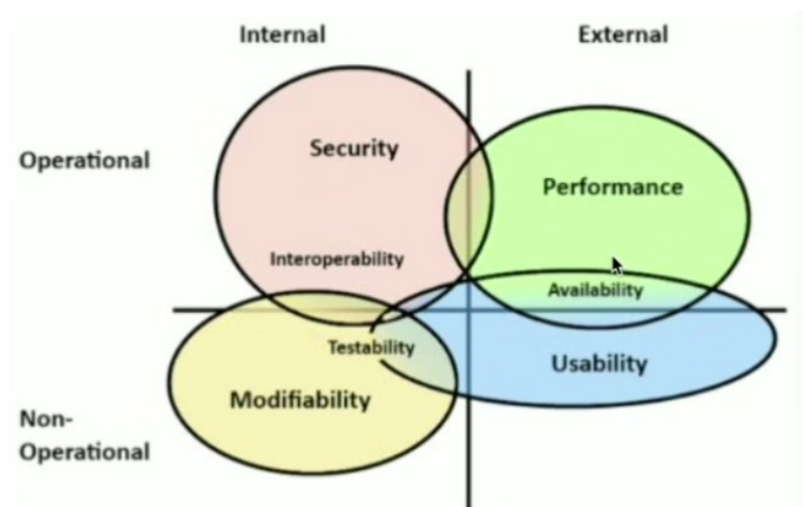
ATRIBUTOS DE CALIDAD

Es una medida o propiedad testeable de un sistema que es usada para indicar cómo el sistema satisface las necesidades de sus Stakeholders.

El usuario genera un estímulo, se genera un artefacto de ese estímulo, y como respuesta se generan métricas.



CLASIFICACIÓN



Otra clasificación puede ser la siguiente:



¿POR QUÉ SE REDISEÑAN LOS SISTEMAS?

Dificultades de mantenimiento; falta de portabilidad; problemas de escalabilidad (muchos usuarios); problemas de performance; problemas de seguridad; interfaces gráficas obsoletas; etc.

Las consideraciones de calidad se desprenden de las necesidades del negocio y deben jugar un rol fundamental durante el ciclo de vida del desarrollo de software. No todo es funcionalidad en un sistema.

Los atributos de calidad deben ser considerados a lo largo de todo el ciclo de vida del software para tener éxito: no dependen solamente de la etapa de diseño (y/o arquitectura)

EJEMPLOS DE ATRIBUTOS DE CALIDAD: Disponibilidad (disponible todo el tiempo, que no esté caído), reusabilidad, performance (velocidad de respuesta del sistema), escalabilidad (que el sistema pueda soportar grandes cantidades de usuarios), testeabilidad (el sistema debe poder ser testeable), interoperabilidad (que pueda comunicarse con otros sistemas externos), usabilidad (fácil e intuitivo de usar), mantenibilidad (capacidad de ser modificado de forma eficiente y efectiva, y que se mantenga en el tiempo), integridad (que no se pierdan datos), portabilidad (que pueda correrse en otras plataformas), confiabilidad (confiar en el sistema, confiar en qué todo va a funcionar como debería).

Los atributos de calidad pueden entrar en conflicto unos con otros (por ejemplo, seguridad vs confiabilidad, portabilidad vs performance, seguridad vs usabilidad, performance vs modificabilidad). El objetivo es evaluar cuantitativa y/o cualitativamente múltiples atributos de calidad: consensuar una priorización de los atributos de calidad con los intereses involucrados; diseñar un sistema lo suficientemente bueno para todos los interesados.

El cliente va a definir que atributos son aquellos a los que se les dará más importancia, y a estos se les pondrá una métrica y en qué casos se aplicará. Esto es así porque los atributos pueden entrar en conflicto unos con otros (por ejemplo:

DISPONIBILIDAD: se refiere a la capacidad del sistema para estar en funcionamiento y accesible cuando se le necesita. En otras palabras, se trata de la capacidad de un sistema o aplicación de estar y operativo para

los usuarios durante un período de tiempo especificado. Se mide generalmente en términos de porcentaje, que representa la proporción del tiempo en el que el sistema está operativo en relación con el tiempo total. Los factores clave que influyen son: un diseño robusto, la gestión de recursos, el mantenimiento y las actualizaciones, la supervisión y la detección de fallos, la recuperación ante desastres, entre otros. Es esencial para garantizar que los sistemas de software cumplan con las expectativas de los usuarios y las necesidades del negocio.

PERFORMANCE: corresponde a los tiempos de respuesta de la aplicación en relación a las funcionalidades o actividades soportadas por la misma. Hay dos formas para medir el rendimiento: la latencia, que es el tiempo dedicado a responder a un evento, y la capacidad, que es el número de eventos que pueden ocurrir en un tiempo determinado.

INTEROPERABILIDAD: mide la capacidad de intercambio de información de la aplicación con otros sistemas o con el entorno donde opera. Una aplicación bien diseñada facilita la integración con otros sistemas. Para mejorar la interoperabilidad, es conveniente utilizar interfaces externas bien diseñadas, normas de intercambio y estándares, entre otras.

USABILIDAD: se puede ver a través de: comprensibilidad (refleja qué tan fácil es para el usuario comprender el sistema, que conocimientos previos requiere el usuario para poder trabajar con el software), fácil uso/eficiente (que permita realizar las operaciones de manera rápida y efectiva), fácil de recordar/intuitiva/estándar, atractividad/agradable/cómoda.

SEGURIDAD: permite medir la vulnerabilidad de las aplicaciones a ataques accidentales o maliciosos, versus la probabilidad de defensa del sistema ante pérdidas o robo de información estratégica y valiosa para la organización. Estos ataques pueden ser tanto externos como internos. Autenticación, autorización, encriptación de datos, auditoría, etc.

- Capacidad de detección de ataques de denegación de servicio (DDoS) y respuesta ante estos.
- Restricciones de acceso de usuarios de acuerdo a las políticas de autenticación y autorización.
- Prevención de la inyección de consultas SQL.
- Encriptación de claves, contenidos y datos empresariales.
- Conexión segura.

ESCALABILIDAD: capacidad de manejar la carga de trabajo de la aplicación sin afectar el rendimiento de esta, es la posibilidad de crecimiento sin perjudicar su funcionamiento operativo. ¿Cómo mejorar la escalabilidad?

- VERTICAL: para crecer, se agregan más recursos físicos a la infraestructura que soporta al aplicativo, tales como memoria, almacenamiento en disco, procesador o capacidad de cómputo, ancho de banda, entre otras, para un aplicativo.
- HORIZONTAL: se incrementa el número de computadoras para dividir la carga de trabajo de la aplicación.

MODELO DE VISTAS DE ARQUITECTURA

Son los conceptos o propiedades fundamentales de un sistema en su entorno encarnado en sus elementos, relaciones y en los principios de su diseño y evolución. Representa la estructura o las estructuras del sistema, que consta de componentes de software, las propiedades visibles externamente y las relaciones entre ellas.

Martin Fowler: "La arquitectura de software son aquellas decisiones que son importantes y difíciles de cambiar.

¿Cómo contamos la arquitectura de un sistema? ¿Cómo diseñamos una posible arquitectura para un sistema como para hacer una prueba de concepto? ¿Cómo documentamos la arquitectura de un sistema? ¿Cómo pensamos un documento de arquitectura de un sistema? Estas son preguntas que hay que hacerse cuando se habla con el cliente (parecido al Product Discovery).

¿Qué debemos tener en cuenta cuando escribimos un documento, un informe o queremos transmitir alguna información? A quién va dirigido, y ahí vamos a analizar qué y cómo le contaremos la idea. Dentro de un equipo podemos encontrar: clientes, usuarios, líderes de proyectos, Product Owners, arquitectos, analistas, Product Designers, UX Researcher, UI Designer, UX Writer, Scrum Masters, desarrolladores, Testers, administradores de bases de datos, Devops, gente de infraestructura o despliegue, etc.

En realidad, existen muchos interesados o Stakeholders de un documento.

MODELO DE VISTAS 4+1

VISTA LÓGICA: apoya principalmente los requisitos funcionales (lo que el sistema debe brindar en términos de servicios a sus usuarios). Aquí se aplican los principios de abstracción, encapsulamiento y herencia. Esta descomposición no sólo se hace para potenciar el análisis funcional, sino también sirve para identificar mecanismos y elementos de diseño comunes a diversas partes del sistema.

VISTA DE DESARROLLO (O DE COMPONENTES): se centra en la organización real de los módulos de software en el ambiente de desarrollo del software. El software se empaqueta en partes pequeñas (bibliotecas de

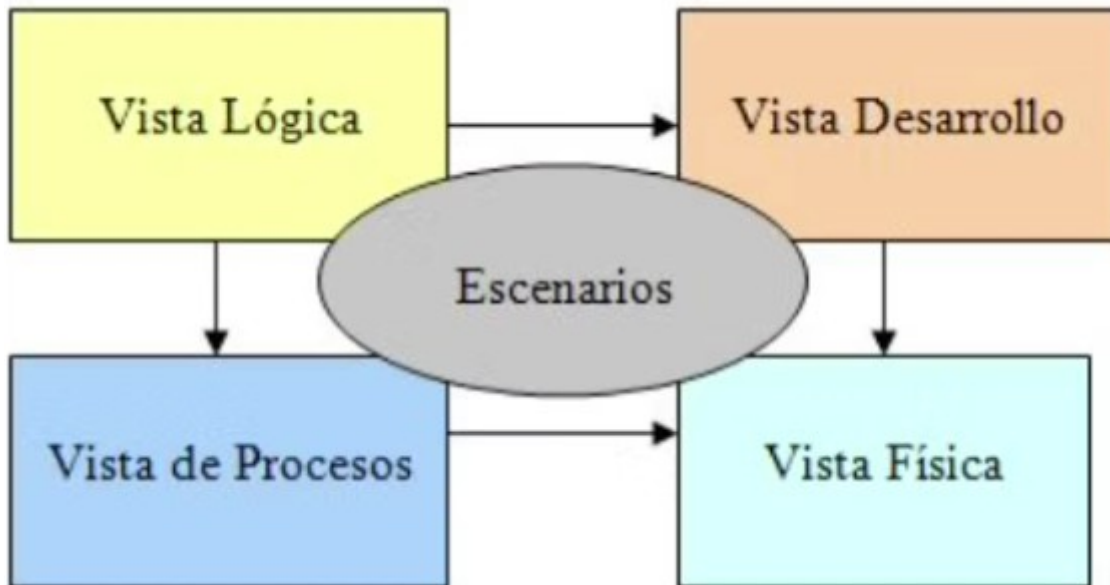
programas o subsistemas) que pueden ser desarrollados por uno o un grupo pequeño de desarrolladores. Esta vista tiene en cuenta los requisitos internos relativos a la facilidad de desarrollo, administración del software, reutilización y elementos comunes, y restricciones impuestas por las herramientas o el lenguaje de programación que se use.

VISTA FÍSICA (O DE DESPLIEGUE): toma en cuenta primeramente los requisitos no funcionales del sistema tales como la disponibilidad, confiabilidad (tolerancia a fallas), rendimiento (throughput) y la escalabilidad. El software se ejecuta sobre una red de computadores o nodos de procesamiento. Los variados elementos identificados (redes, procesos, tareas y objetos) requieren ser mapeados sobre los nodos. Esperamos que diferentes configuraciones puedan usarse: algunas para desarrollo y pruebas, otras para mostrar el sistema en varios sitios para distintos usuarios. Por lo tanto, la relación del software en los nodos debe ser altamente flexible y tener un impacto mínimo sobre el código fuente.

VISTA DE PROCESOS: toma en cuenta algunos requisitos no funcionales tales como el rendimiento y la disponibilidad. Se enfoca en asuntos de concurrencia y distribución, integridad del sistema, de tolerancia a fallas. Un proceso es una agrupación de tareas que forman una unidad ejecutable. Los procesos representan el nivel al que la arquitectura de procesos puede ser controlada tácticamente (i.e., comenzar, recuperar, reconfigurar y detener). Además, los procesos pueden replicarse para aumentar la distribución de la carga de procesamiento, o para mejorar la disponibilidad.

Todo lo que se hace en estas vistas, es para darle forma a los escenarios, que es la “quinta” vista. Son los casos de uso.

La vista lógica son los conceptos lógicos que yo pienso necesarios para resolver los escenarios. La vista de desarrollo son componentes que yo pienso para empaquetar lo lógico para poder dar soporte a los escenarios (lo mismo para la vista de procesos y la física).



VISTA DE ESCENARIOS: los elementos de las cuatro vistas trabajan conjuntamente en forma natural mediante el uso de un conjunto pequeño de escenarios relevantes. Estos son de alguna manera una abstracción de los requisitos más importantes. Sirve a dos propósitos principales: como una guía para descubrir elementos arquitectónicos durante el diseño de arquitectura; y como un rol de validación e ilustración después de completar el diseño de arquitectura, en el papel y como punto de partida de las pruebas de un prototipo de la arquitectura.

CONCLUSIONES:

- Permite a través de diferentes vistas analizar distintas perspectivas del problema, focalizándose en el problema en cuestión.
- Concentra en un único documento las principales decisiones tomadas sobre el sistema.
- Permite a nuevos integrantes del equipo entender la arquitectura del sistema y ubicarse dentro de la solución.
- Permite discutir con todos los stakeholders las distintas decisiones y validarlas en una etapa temprana.

DECISIONES DE ARQUITECTURA

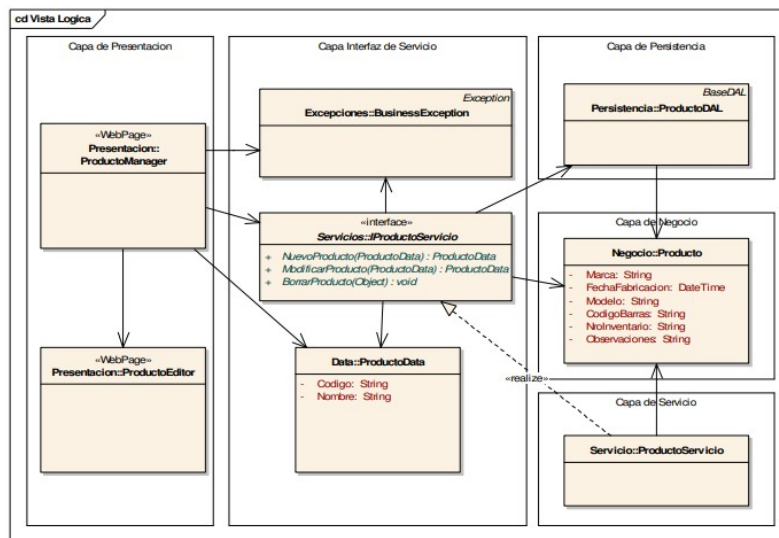
Son decisiones de diseño que abordan requisitos significativos desde el punto de vista arquitectónico; se perciben como difíciles de hacer y/o costosos de cambiar. Estas decisiones son importantes de guardarlas y

conocerlas. Conocer el motivo por el cual se tomó dicha decisión en la arquitectura o diseño.

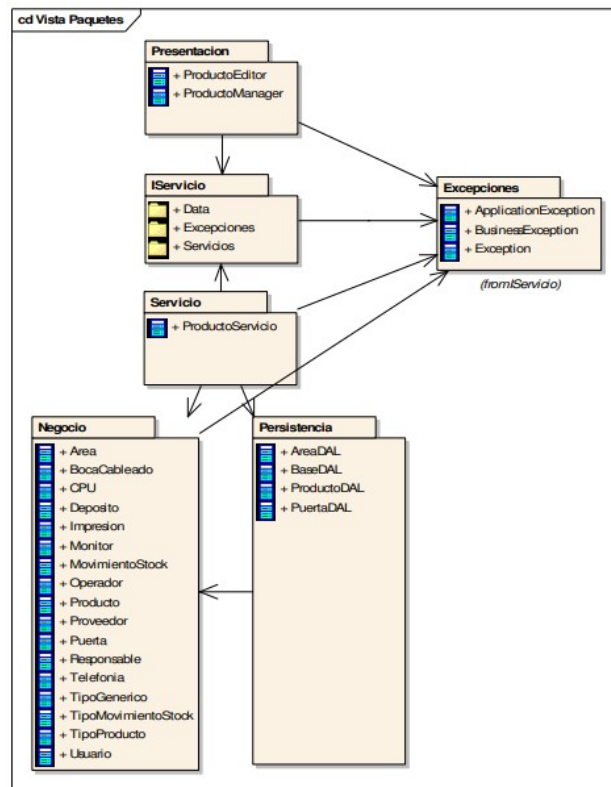
Por ejemplo, para el envío de emails se usa un SMTP y realiza el envío programáticamente gestionado por nuestro backend o se delega a un servicio de *email provider* de terceros como Sendgrid, Nailgun, etc.

EJEMPLO ARQUITECTURA

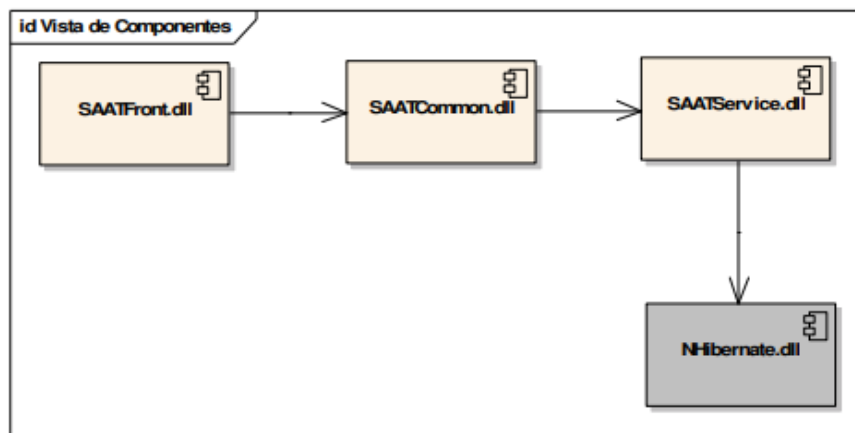
VISTA LÓGICA



Vista de paquetes:



Vista de componentes:

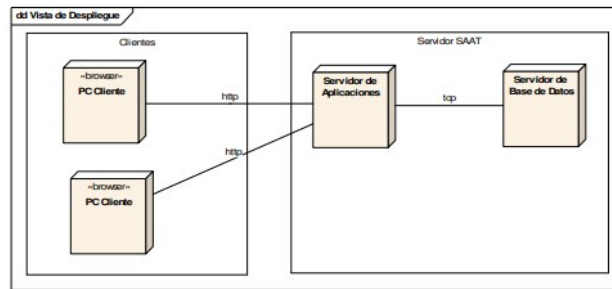


Los componentes principales son:

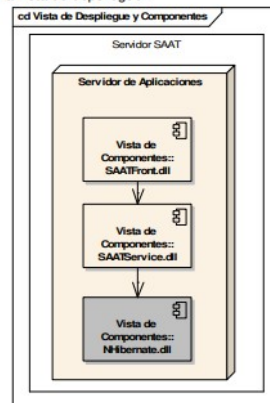
- **SAATFront:** Agrupa las clases encargadas de mostrar la información y permitir interactuar al usuario con la aplicación. Define las pantallas a mostrar según el estado de ejecución, controla la navegabilidad del sistema. Formatea los datos de salida. Valida el formato de los datos de entrada para permitir que el usuario pueda enterarse de las validaciones antes de que su solicitud llegue a la capa de lógica de negocios.
- **SAATService:** Agrupa las clases que definen las reglas de negocio. Publica los servicios de negocio y controla su acceso. Incluye las clases de manejo de persistencia.

Cada componente es un proyecto, es decir, un ejecutable.

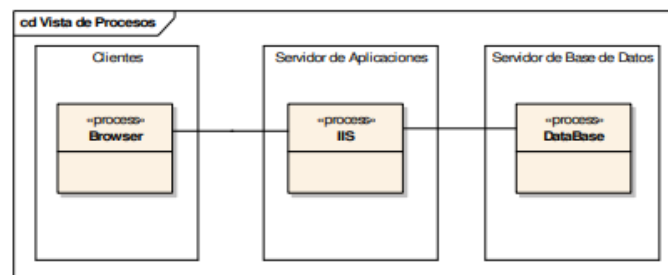
VISTA FÍSICA



A continuación se muestra como se despliegan los componentes físicos dentro de los servidores analizados en la vista de despliegue.



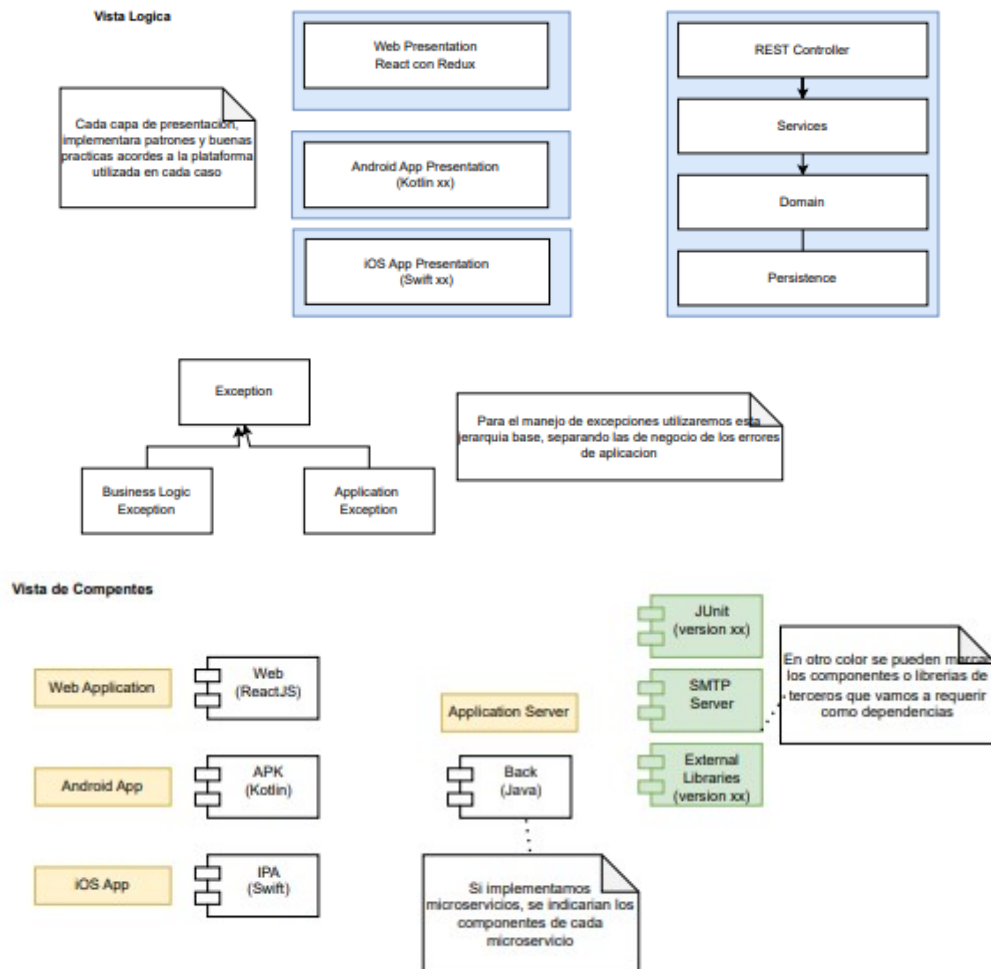
VISTA DE PROCESOS



En el diagrama se pueden observar los procesos que son necesarios para el funcionamiento de la aplicación SAAT.

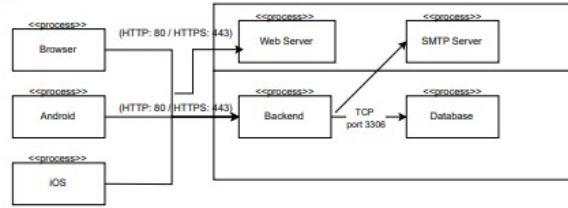
Tenemos un primer proceso, llamado Browser, el cual representa al proceso de los browsers clientes que acceden a la aplicación, comunicándose con el proceso llamado IIS, el cual hostea todos los componentes de la aplicación SAAT. Existe otro proceso, llamado DataBase Process, que refleja el proceso del servidor de la base de datos relacional.

OTRO EJEMPLO:

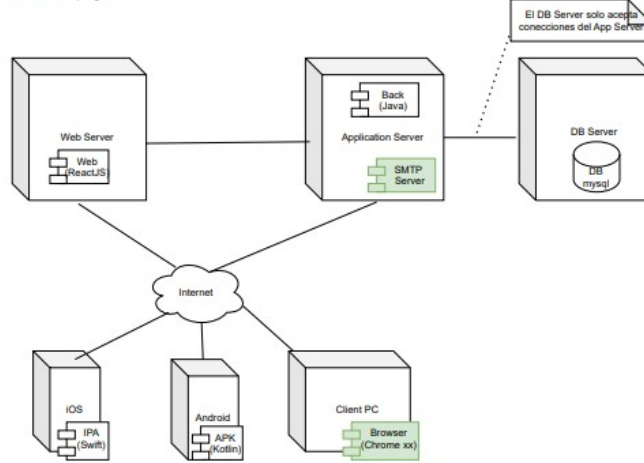


Lo azul son dependencias que luego se resolverán. Cada uno de los nodos es un ejecutable

Vista de Procesos



Vista de Despliegue



PATRONES DE ARQUITECTURA

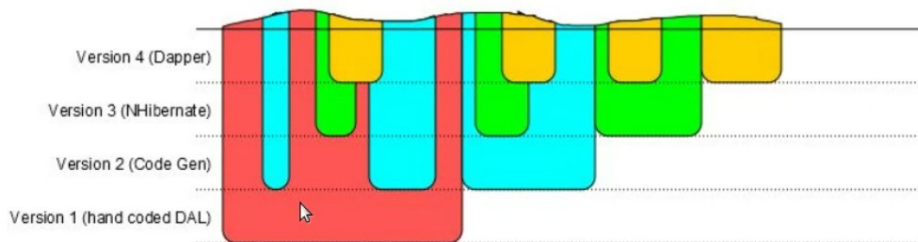
“La arquitectura de software son aquellas decisiones que son importantes y difíciles de cambiar” – Martin Fowler

Según Wikipedia: los patrones de diseño son técnicas para resolver problemas comunes; un antipatrón de diseño es un patrón de diseño que invariablemente conduce a una mala solución para un problema.

ANTIPATRONES

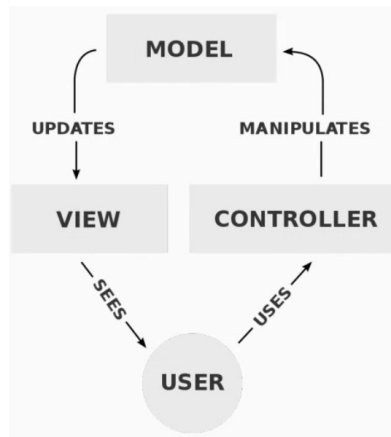
GRAN BOLA DE LODO: Estructura erosionada o que nunca existió. Acción por cambios impredecibles hace difícil reestablecer estructura. Es la versión de código espagueti para la arquitectura. El problema que tiene esto es que con el tiempo, al no haber organizado el código desde un principio, se vuelve muy difícil leerlo y entenderlo (no es mentnible).

LAVA LAYER: cambios sucesivos + no modificar código viejo. Hace falta considera la arquitectura en cada etapa. Básicamente, hay que intentar mantener todas las estructuras al día.



BUENOS PATRONES

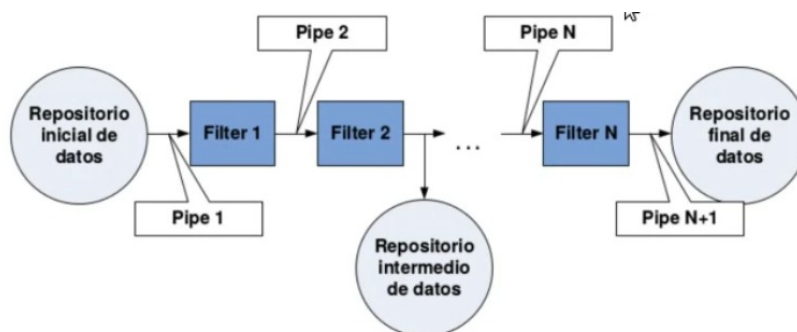
MODEL-VIEW-CONTROLLER (MVC): tres roles distinguidos: modelo (objeto invisible que representa información sobre el dominio modelado, y se comporta según las reglas intrínsecas del negocio), vista (una forma de representar el estado del modelo de manera que sea perceptible, y solo muestra información) y controlador (encargado de tomar los estímulos recibidos y provocar las perturbaciones correspondientes al modelo). La importancia de la separación radica en: los tres roles son incumbencias separadas; múltiples vistas posibles para percibir el modelo; múltiples controladores permiten diferentes interacciones con el modelo; objetos de modelo más fácilmente testeables.

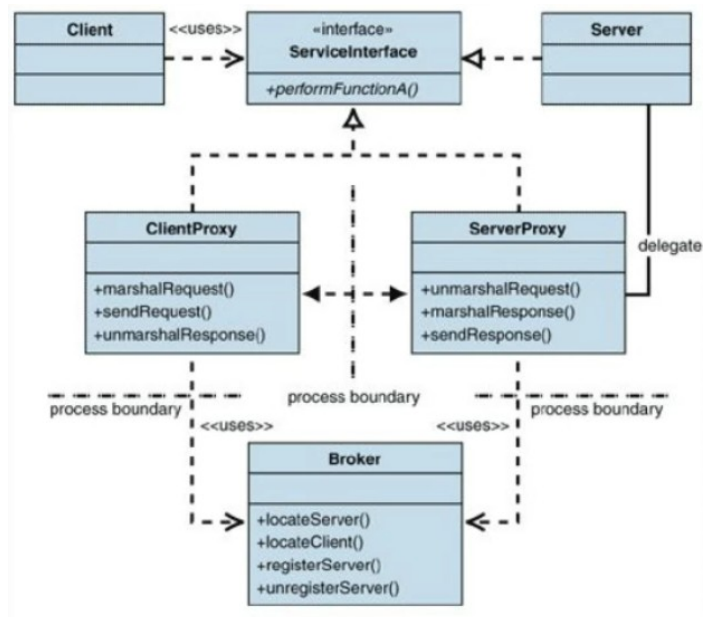


LAYERS: cada nivel recibe servicios del que tiene debajo y provee servicios a la que tiene arriba. La interacción es entre capas adyacentes. Los beneficios son: las capas son más simples que el todo, independientes/desacopladas, intercambiables, estandarizables y reusables. Las contras son: no se puede encapsular todo; cambios de cascada, la interacción entre capas implica overhead; se debe cuidar la cantidad; no es trivial definir la división más correcta.

PIPE & FILTER: un pipe es una conexión entre dos filters o repositorios; un filter es lo que mueve datos entre pipes, usualmente con alguna modificación. Se usa cuando queremos encapsular distintas tareas sobre un mismo conjunto de datos en componentes individuales, cada uno encargado de realizar una tarea específica. También cuando necesitamos poder interconectar dichos componentes, o cuando queremos poder construir distintas “líneas” de procesamiento interconectando componentes reutilizables.

Algunos usos conocidos son: para comandos consola, filtros HTTP, compiladores (cada etapa de procesamiento se comporta como un filtro), workflows, etc.





Algunos Brokers conocidos son: CORBA (Common Object Request Broker Architecture), RMI, .NET Remoting, Active MQ, etc.

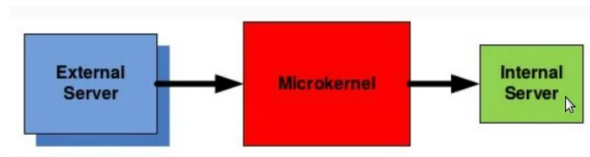
ENTERPRISE ARCHITECTURE: una aplicación Enterprise es un sistema fuertemente orientado a un negocio determinado, que debe cumplir ciertos requerimientos no funcionales.

- RESPONDE A CIERTAS NECESIDADES: codificar las reglas de negocio; representar las entidades de negocio; persistir su estado; garantizar la coherencia de datos; manejar la distribución de la aplicación.
- CARACTERÍSTICAS HABITUALES: lógica de negocio; persistencia de datos en forma masiva; multiusuarios; múltiples interfaces de usuario; disonancia conceptual (no tiene un concepto central, depende de las reglas de negocio).

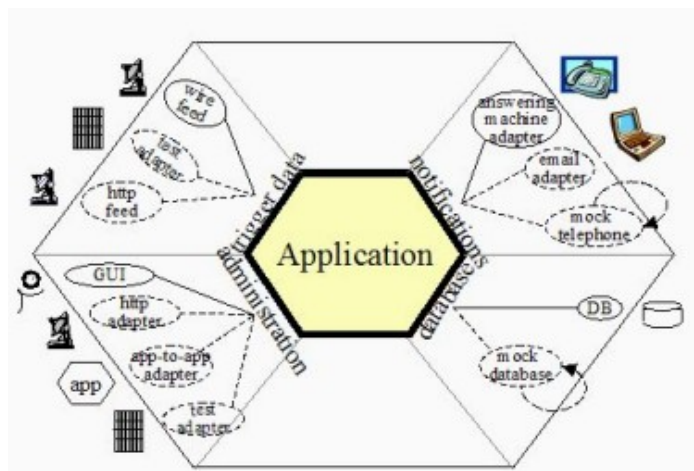
ENTERPRISE ARCHITECTURE + LAYERS: el *Enterprise* architecture también se combina con el modelo de capas: presentación, dominio y persistencia. Cada capa está aislada de forma lógica y física (en diferentes máquinas. Por ejemplo, Frontend (presentación), Backend (controlador, servicio: casos de uso, dominio: modelo de objetos, acceso a datos) y base de datos (persistencia)

MICROKERNEL: kernel significa núcleo. Separa el sistema en un núcleo y componentes que usan/proveen servicios. Los servidores externos usan servicios del núcleo, y el núcleo se basa en los servidores internos para poder proveerlos. Se usa en sistemas de larga vida que deben evolucionar ante cambios de distinto tipo y con diferentes frecuencias. Tiene como

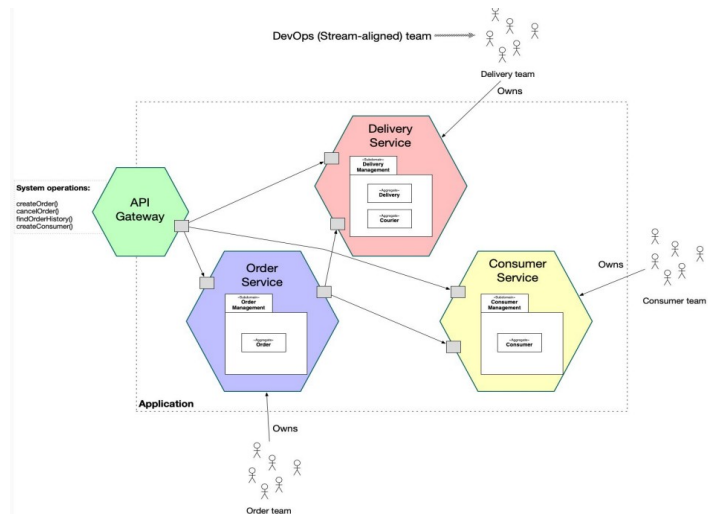
ventaja el soportar cambios con menor impacto, aunque una desventaja es que tiene un diseño más elaborado.



ARQUITECTURA HEXAGONAL: hay una parte central (la que habla del dominio, donde está la lógica de negocio), y por fuera se le proveen diferentes servicios. La idea es tener un núcleo que sea estable a lo largo del tiempo, y dejar que toda la parte de infraestructura quede por fuera.



MICROSERVICIOS: la idea principal es separar la aplicación en varios microservicios. Desde el punto de vista de cada uno de los microservicios, los otros servicios son dependencias externas que son manejadas por otro equipo. La complejidad entera del dominio va a suceder en la interacción entre los distintos servicios. Las ventajas que tiene este patrón es su escalabilidad, reusabilidad y aislamiento en cuanto a datos, tecnología y conceptualmente. Sin embargo, una desventaja es su complejidad. Si hay acoplamiento entre microservicios (protocolos de comunicación, librerías de código, estructura de base de datos, otros microservicios), va a generar que se va a tener que trabajar con todo el sistema a la vez



RESTFUL API

No es un standard, ni un protocolo, ni una librería. Se basa en usar estándares existentes como HTTP para facilitar la comunicación entre el cliente y el servidor. Se presenta en 3 niveles de madurez.

CARACTERÍSTICAS

STATELESS: cada request (petición) se ejecuta de forma independiente, y cada una contiene toda la información necesaria para completarse. La API no mantiene ningún tipo de sesión (mejora la escalabilidad porque permite que un cliente sea atendido por distintos servidores). Se promueve el uso de tokens para manejo de seguridad. No almacenar información del cliente en el servidor, en todo caso el cliente le volverá a mandar la información necesaria.

CACHEABLE: permite reducir el ancho de banda usado, la latencia y reducir la carga en servidores. También permite ocultar los fallos de red. Lo que se define como “cacheabilidad” en los sistemas REST es la capacidad de estos sistemas para etiquetar de alguna forma las respuestas para que otros mecanismos intermedios funcionen como un caché. Estos sistemas o mecanismos intermedios (existen entre el cliente y el servidor) deben ser por lo general transparentes para los desarrolladores, no deben afectar la manera en que los servicios se consumen.



COMPRESIÓN: las APIs suelen retornar representaciones en varios formatos, y estos pueden ser comprimidos para ahorrar anchos de banda

Accept-Encoding

Ejemplo de cómo el cliente informa que mecanismos soporta:

```
Accept-Encoding: gzip,compress
```

Content-Encoding

Ejemplo de cómo el server informa que mecanismo usó para encriptación:

```
Content-Type: text/html  
Content-Encoding: gzip
```

Ejemplo Node

sobre la red.

```
const compression = require('compression');  
app.use(compression()); // activa gzip si el cliente lo soporta
```

URIS: *Uniform Resource Identifier*, identificación unívoca de recursos con cadenas de caracteres. Identifica los recursos por clase o tipo. Usa sustantivos en plural por convención, y hay una distinción de recursos principales y subordinados.

Ejemplos: /cliente (representa todos los clientes), /clientes/1 (representa al cliente con id 1), /clientes?nombre=juan (representa a los clientes con nombre juan), /clientes/1/compras (representa a las compras del cliente 1)

¿Si las compras fuesen recursos primarios? /compras, /compras?cliente=1.

VERBOS HTTPS – REQUESTS

GET: solicita una representación del recurso específico

POST: se utiliza para enviar una entidad a un recurso específico

DELETE: borra un recurso específico

PUT: reemplaza todas las representaciones actuales del recurso de destino con la carga útil de la petición

PATCH: aplica modificaciones parciales a un recurso (a diferencia de PUT)

OPTIONS: es utilizado para describir las opciones de comunicación para el recurso de destino.

HTTP STATUS CODES

1xx: informational

2xx: success (200 – OK, 201 – Created)

3xx: redirection

4xx: client error (400 – Bad Request, 401 – Authorization Required, 404 – Not Found, 405 – Method Not Allowed, 408 – Request Time-Out, 409 – Conflict, 422 – Unprocessable Entity)

5xx: server error (500 – Internal Server Error, 502 – Bad Gateway, 504 – Gateway Time-Out)

REST SECURITY DESIGN PRINCIPLES

LEAST PRIVILEGE (tener el menor privilegio requerido para hacer acciones), FAIL-SAFE DEFAULTS (por defecto no tener acceso a los recursos), COMPLETE MEDIATION (el sistema debe validar los permisos de acceso a todos los recursos), KEEP IT SIMPLE, HTTPS, PASSWORD HASHES, NEVER EXPOSE INFORMATION ON URLS (usernames, passwords, session tokens y API keys deberían no aparecer en la URL para evitar ser logueadas en los logs de web server logs), CONSIDERAR AGREGAR TIMESTAMP EN LOS REQUEST, VALIDACIÓN DE LOS PARÁMETROS DE ENTRADA

MONITOREAR TRANSACCIONES SOSPECHOSAS: cantidad de requests por IP o por token/JWT/user para evitar problemas de denegación de servicio, o simplemente controlar o reducir el uso excesivo que puede bajar la performance de la API en general. Limitación de velocidad o tiempos de demora agregados entre request y request para ciertos casos, ayuda a reducir las solicitudes excesivas que ralentizarían la API, ayuda a lidiar con llamadas/ejecuciones accidentales y monitorea e identifica de manera proactiva una posible actividad maliciosa.

AUTENTICACIÓN Y AUTORIZACIÓN

- **BASIC AUTH:** trabaja con un header HTTP llamado authorization que manda una tira de bytes (<user>:<password>, encodeado en Base64), un user, una password y un plain-auth. El server hace lo contrario, decodificando en Base64. Es un estándar de HTTP. Es una manera de guardar la información en el header.

Base64 encoding

- user: **fiuba**
- pass: **k@X4R\$KFEBcN**
- **plain-auth:** **fiuba:k@X4R\$KFEBcN**
- Authorization: **Zml1YmE6a0BYNFikS0ZFykNu**

- **API KEYS:** es un token (tira de bytes) que el cliente provee cuando hace la llamada vía queryString. Se supone que es secreta y que solo el cliente y el servidor la conocen. Sólo debería usarse en conjunto con otro mecanismo de seguridad como HTTPS/SSL. Generalmente se utilizan del lado del servidor. No es recomendable que vaya en el queryString.

```
1. GET /something?api_key=abcdef12345

Como header:

1. GET /something HTTP/1.1
2. X-API-Key: abcdef12345

Como cookie:

1. GET /something HTTP/1.1
2. Cookie: X-API-KEY=abcdef12345
```

- **BEARER AUTHENTICATION/TOKEN AUTH:** utiliza tokens de seguridad llamados Bearer (da acceso al portador del token). Se envía un header de Authorization: `Authorization: Bearer <token>`. Es muy parecido a una API Key (se manda un header authorization: Bearer <Token>). Es una tira de bytes que se obtiene de una autenticación previa, y es un token dinámico (tiene tiempo de vida). Se autentica una única vez y te genera un authentication token que lo mando, y es el que está asociado al usuario por un tiempo determinado.
- **OAUTH:** es un protocolo de autenticación que consiste en delegar la autenticación de usuario al servicio que gestiona las cuentas, de modo que sea éste quien otorgue el acceso para las aplicaciones de terceros. OAuth 2 provee un flujo de autorización para aplicaciones web, aplicaciones móviles e incluso programas de escritorio.

- | HEADER:ALGORITHM & TOKEN TYPE | PAYLOAD:DATA | VERIFY SIGNATURE |
|---|---|---|
| <pre>{ "alg": "HS256", "typ": "JWT" }</pre> | <pre>{ "uid": 1234567890, "name": "John Doe", "prof": "OWNER", "org": 8347523 }</pre> | <pre>HMACSHA256 (base64UrlEncode(header) + "." + base64UrlEncode(payload), hqo245thSD\$KfgJtgu#%ebvjqf?2f3idJjdivl)</pre> |

Los JWT pueden ser mensajes solo firmados, solo encriptados, o ambos. Si un token es solo firmado, pero no encriptado, cualquiera puede leer su contenido, pero si no se conoce la clave privada no puede modificarlo ya que al validar la firma no coincidiría.



El server valida la firma del JWT para saber que lo que él le envió al cliente no se modificó, y utiliza la información de este. De esta forma se sigue siendo stateless, y generalmente se hace más eficiente la validación del usuario, sin tener que acceder a un medio persistente a validar si el token es válido y a quién pertenece el mismo.



- **REFRESH TOKENS:** los access token (token que genera JWT) deberían tener un tiempo limitado de vida, por eso aparecen los refresh token. Esto es otro token que sirve para un solo uso y es utilizado para obtener un nuevo access token. Se almacena en la

base de datos, y cuando uno se logea, recibe el access y el refresh token. El refresh lo guarda y no lo usa más hasta que, usando el access token, la API le devuelve que este está vencido, y es ahí cuando agarra el refresh token, que tiene un tiempo de vida mayor, y llama a otro servicio que le devuelve nuevos access y refresh tokens. En otras palabras, es una credencial que permite obtener nuevos tokens sin necesidad de usar las credenciales de user y password nuevamente.

VERSIONADO

REST no provee un mecanismo definido para versionado, pero se suelen ver estas estrategias:



Usando la URI:

- `https://api.fi.uba.ar/v1`
- `https://api.v1.fi.uba.ar`
- `https://api.fi.uba.ar/20211101/`

Usando un Custom Header:

- `Accept-version: v1`
- `Accept-version: v2`

Usando el Header Accept:

- `Accept: application/vnd.example.v1+json`
- `Accept: application/vnd.example+json;version=1.0`

RESPUESTAS

Mantener las respuestas lo más estándar posible. Reducir su tamaño a lo necesario, y utilizar códigos de errores HTTP.

```
HTTP CODE: 401 {
  "success": false, // solo informativo, el error se define por el HTTP CODE
  "message": "Invalid email or password",
  "error_code": 1308,
  "data": {}
}
HTTP CODE: 200 {
  "success": true,
  "message": "User logged in successfully", // optional in success responses
  "data": { }
}
```

Donde data es un objeto que contiene un mapa de otros objetos